Vector Bin Packing Problem

1. Abstract

The Vector Bin Packing Problem (VBPP) extends the classical bin packing problem to instances of multiple dimensions and is applicable to uses such as resource allocation for VMs. As an NP-hard problem, practical solutions rely on heuristics. This project evaluates a range of heuristics, including existing methods and some newly introduced heuristics, testing on datasets with various dimensions and structure. Performance is assessed in terms of quality of packing, computation time, and reliability, with a focus on item ordering strategies using different measures of size.

2. Table of Contents

1. Abstract 1
3. Introduction 3
4. Aims, Objectives, and Requirements 4
5. Literature Review 4
5.1 Nature of the Problem 4
5.2 Testing Methods 5
5.3 Existing Heuristics 6
5.4 Summary 8
6. Design 9
6.1 Key Ideas 9
6.1.1 Measures of Size 9
6.1.2 Sorting Algorithm 13
6.1.3 Benchmark Data 14
6.2 Heuristics 15
6.2.1 First Fit 15

6.2.2 Item-Centric FFD 15
6.2.3 Bin-Centric FFD 16
6.2.4 Grouped by Most Significant 16
6.2.5 First N Items 16
6.2.6 Decaying Lookahead 17
7. Implementation 19
7.1 Framework 19
7.1.1 main.cpp 19
7.1.2 mergeSort.cpp 21
7.2 Heuristics 26
7.2.1 Item-Centric First Fit (Decreasing) 26
7.2.2 Bin-Centric FFD 27
7.2.3 Dot-Product Bin-Centric FFD (Panigrahy, et al., 2011) 28
7.2.4 Grouped By Most Significant 29
7.2.5 First N Items 31
7.2.6 Decaying Lookahead 32
8. Evaluation of Implemented Heuristics 35
8.1 Sorting Methods 35
8.1.1 2CBP 35
8.1.2 Panigrahy 39
8.1.3 Mommessin 45
8.1.4 Triplet 47
8.2 Item-Centric vs Bin-Centric 48
8.3 Comparison of Heuristics 50
8.3.1 2CBP 50
8.3.2 Panigrahy 56

8.3.3 Mommessin 64

8.3.4 Triplet 67

9. Conclusion 69

10. Final Comments 71

11. Bibliography 72

3. Introduction

Resource allocation, particularly for virtual machines, is a fundamental and increasingly important problem in computer science. As cloud computing is more relevant than ever, and only increasing in use, the need for efficient resource allocation is crucial to minimise costs and to extract maximum utilisation out of each physical device. One of the most widely known optimisation problems is bin packing, and relating to resource allocation, the vector bin packing problem.

In this project, the focus will be on the vector bin packing problem, and some of the heuristics used to find solutions to the problem. The VBPP is a more advanced version of the bin packing problem, which fits scalar items into fixed sized containers, using as few containers as possible. Instead of scalars, the VBPP uses d-dimensional vector items, to fit into d-dimensional bins. This is harder than the standard bin packing problem as there is no standardised way to compare the size of items like there is with a scalar. Many norms, such as L_1, L_2, and L_∞ can be used to determine the size of a vector, as well as some which consider the remaining capacities of open bins.

The VBPP is more relevant to resource allocation than the classical bin packing problem, as very rarely is only one resource required. When allocating resources for a virtual machine, the requirements may include CPU cores, RAM, Disk Space, and GPU Cores. Splitting these across multiple servers may not always be practical, therefore this can be seen as the VBPP.

While the problem definition is simple, both problems are NP-Hard, and very difficult to find optimal solutions for when there is a large input set, meaning heuristics are often used to

find an acceptable solution in reasonable time. The offline variant of the problem has a fully known dataset from the start, whereas the online variant receives items one at a time. This project will be looking into the offline variant of the problem. This is because knowing each item in advance can yield better results, with the online variant requiring immediate and permanent allocation decisions to be made. While the online variant of the problem is more applicable to real-time assignments on a cloud platform, the offline variant is applicable to cases where resources need to be allocated in bulk, or migration of virtual machines.

As part of the project, some existing and new heuristics will be implemented, tested, and compared, before a final evaluation of the results is carried out.

4. Aims, Objectives, and Requirements

Set out below is a list of goals and criteria for this project:

- To critically analyse the nature of the VBPP and how it can be defined.
- To research existing heuristics and outline the logic behind them.
- To define and analyse different ways to approach the problem.
- Implement a variety of existing heuristics and algorithms, including different measures of size of vectors.
- Design, implement, and refine some new heuristics.
- Implement and evaluate different ways of ordering items using existing and newly introduced measures of size.
- Evaluate the effectiveness of each heuristic on a variety of benchmark datasets.
- Compare the implemented heuristics and synthesise a conclusion to recommend which may be more appropriate in certain types of datasets or use cases.

5. Literature Review

5.1 Nature of the Problem

The bin packing problem, and more specifically vector bin packing, is a well-researched problem with a wide range of academic papers and research on the nature of the problem, various different ways to find acceptable solutions to the problem in many different cases,

and analysis of why certain heuristics work in some cases better than others. It was first described by Garey et al. (Garey, et al., 1976) in 1976.

The multidimensional VBP problem, as described in (Csirik, et al., 1990), takes a list of n items:

$$L = (x_1, x_2, \dots, x_n)$$

Where each item is a vector of form:

$$\big(s_1(x_i), s_2(x_i), \dots, s_m(x_i)\big), 1 \leq i \leq n$$

In most formal definitions of the problem, bin size must be 1 in all dimensions, and therefore x_i must have no dimensions that exceed the size of 1. However, some definitions (Mommessin, et al., 2025) do not specify the bin size, other than that it must be uniform across all bins. This seems appropriate as it is more applicable to real world applications, as it removes the need to normalise data before processing it, but the definition of the problem is still almost the same. In this project, the latter will be used, but with the addition that the capacity c_j of each dimension of the bins must be the same. This means that the problem can be tackled in the same way as the more accepted $0 \leq s_j(x_i) \leq 1$, but with less data pre-processing requirements.

The justification for using heuristics to find a solution to VBP problems is that they are NP-hard meaning the problem is at least as hard as the hardest NP-complete problem. If $P \neq NP$, then NP-hard problems cannot be solved in polynomial time. The VBP problem is a generalisation of the classical bin packing problem (which is known to be NP-complete) as the classical problem can be treated as VBP for 1-dimensional vectors (Spieksma, 1994).

5.2 Testing Methods

There are many benchmark instances used to test how effective a heuristic is in different cases. For 2-dimensional cases, the benchmark “Two-Constraint Bin Packing Problem” from https://site.unibo.it/operations-research/en/research/library-of-codes-and-instances-1 containing instances based on (Spieksma, 1994) and (Capara & Toth, 2001) will be used. For higher dimensional cases, the 3 types of benchmark instance from vectorpack (Mommessin, et al., 2025) are to be used. There are 2 sets based on (Panigrahy, et al., 2011), (Capara &

Toth, 2001) and (Falkenauer, 1996); and one set of newly introduced classes. There are instances with 3, 5, and 10 dimensions, which helps simulate real world requirements, where the number of constraints is not always the same. Across the 4 sets of benchmark instances, there should be enough variety to get a good idea of which heuristics perform well, and when certain heuristics are suitable.

One good measure of how optimal a result is, is the lower bound. There is no known way to calculate the exact number of minimum bins required, but there are lower bounds such that at least that many bins are required, but possibly more. Because of this, if the resulting bin packing from a heuristic uses n bins, and $n = lower\ bound$ then you definitely have the optimal packing. A trivial way of finding this (Spieksma, 1994) is to take the maximum of the sums of each dimension, and divide it by the bin size, i.e.:

$lb = \frac{\max([\sum_{i=1}^{n} s_1(x_i)],[\sum_{i=1}^{n} s_2(x_i)],\dots,[\sum_{i=1}^{n} s_m(x_i)])}{binSize}$ where m is the number of dimensions.

Spieksma also describes a way to calculate the lower bound by pairing items and considering whether items can be paired together and in the same bin. This lower bound is more suitable when there are larger items in the dataset. Therefore, a more accurate lower bound can be found by using the maximum of the lower bounds found by both methods listed.

The lower bound can be useful not only for checking the result, but for determining the nature of the dataset. This information could be used to determine how certain variables and measures are weighted, and which heuristic may be more suitable for the current instance of the problem.

5.3 Existing Heuristics

Many of the heuristics used to find solutions to VBP problems are greedy algorithms, most of which sort the list of items in such a way that they are likely to be packed more optimally than if they had not been sorted. One such algorithm is first-fit decreasing (FFD), which is one of the most widely used heuristics for VBP. There are cases of this being used for VBP as far back as Garey et al. (Garey, et al., 1976) in 1976 and Kou and Markowsky (Kou & Markowsky, 1977) in 1977. Traditionally this approach was item-centric, however Panigrahy et al. (Panigrahy, et al., 2011) used a bin-centric approach with FFD, only opening a new bin

once all existing bins are full and no items fit in them. One way of adapting this algorithm, which I plan to explore, is the measure of size of each item, and therefore the order that items are sorted into.

Another greedy algorithm used for VBP is best-fit decreasing (BFD), and less commonly First-Fit or Best-Fit. The latter two are not often used, as their respective decreasing counterparts perform considerably better. While Best-Fit performs better than First-Fit, BFD performs very similarly to FFD as shown by Li and Chen (Li & Chen, 2006). For that reason, FFD is more often used than BFD due to it being the faster of the two heuristics.

In the bin-centric approach introduced by (Panigrahy, et al., 2011), they introduce a measure of size using the dot product between the vector of remaining capacities of the open bin, and the item vector. This allows their algorithm to find the items that are more likely to fit well in the current bin, rather than just taking the largest item that fits. Relating to the definition of dot product:

$$\cos(\theta) = \frac{a \cdot b}{|a|\,|b|}$$

$$a \cdot b = \cos(\theta)\,|a|\,|b|$$

Given the item is a and the remaining capacity is b, since b will be on a given bin state, we can see that the 2 factors in the "size" of a vector are the angle between the two vectors $\cos(\theta)$ and the magnitude of the item $|a|$. This means the measure of size is essentially the magnitude of the item vector, scaled based on how similar its direction is to that of the remaining capacity of the current bin. In their algorithm, they also include a weight vector used to scale each dimension based on the nature of the dataset, such that if most vectors are bound by the same dimension, that dimension is more significant to in size of every vector. One main idea from both the dot product calculation, and the weight vector is changing the way items are placed based on the properties of the specific instance of the problem, rather than just using a static measure of size for all instances. This idea of altering the size of items based on the current instance will be something that can be explored further and be applied to other heuristics.

Nagel et al. (Nagel, et al., 2023) made use of the previously mentioned bin-centric heuristic in their hybrid heuristic. This works in a similar way but opens a new bin for each item that does not fit in an existing one. Once all items fit in at least one bin, each bin selects an item by either maximising the dot product, or minimising the L_1, L_2, or L_∞ norm of the remaining capacity. Once this has been done, the bin that gets the item it has selected is chosen based on a set of rules. When developing new heuristics, considering multiple options before placing an item could be adapted in various ways to find better solutions than a simple greedy algorithm. Depending on how this is implemented, this could be used to improve existing greedy algorithms, in such a way that makes them slightly slower, but find better results in many cases.

Some more recent heuristics (Nagel, et al., 2023) make use of non-greedy heuristics and employ different ideas to attempt to find a good solution. A non-greedy heuristic means that the placement of a bin is not necessarily final, and can be changed later if it could result in a tighter packing. Because of this, the time complexity of non-greedy heuristics is often worse than greedy heuristics, but can be worth the trade-off if the result is a tighter packing.

One of these heuristics is Local Search (LS)/Local Search with Randomisation (LSR). These heuristics place the largest item, and a randomly selected item into the same bin. It then swaps items randomly if they reduce the remaining capacity. In the LSR heuristic, items can be swapped even if capacity is not reduced as to avoid local optima. The assignment with the lowest remaining capacity is remembered during the run and becomes the chosen solution. Since the use of non-greedy heuristics has been shown to work well in many cases, it will be worth looking into non-greedy heuristics in this project.

5.4 Summary

In summary, the majority of existing heuristics used for the VBP problem are greedy heuristics. Because of this, there is a lot of research already there which can be used as a strong basis to build on. However, there are cases where non-greedy heuristics have been shown to provide better results. Based on this, I will focus on developing both greedy and non-greedy heuristics in my project. The heuristics mentioned above, implemented and tested on the benchmark datasets can be used as a reliable way to measure the effectiveness of the new heuristics.

6. Design

6.1 Key Ideas

6.1.1 Measures of Size

Many heuristics require the items to be ordered by size before they are placed. Unlike with a scalar where the size is clearly measurable and objective, there are many ways to measure the size of a vector due to the multiple dimensions. Some of the most common choices are using the L_1, L_2, and L_∞ norms.

The L_1 norm, or Manhattan Distance, uses the sum of all the elements of the vector. This can be expressed as:

$$s = \sum_{i=1}^{n=d} |v_i|$$

where s is the size of a d-dimensional vector v. This treats every dimension equally in the size of the vector, treating $[10,0,0]$, $[0,10,0]$, and $[5,0,5]$ as equally sized vectors (each of size 10).

The L_2 norm, or Euclidean Distance, uses the sum of the squares of all elements of the vector. This can be expressed as:

$$s = \sum_{i=1}^{n=d} v_i^2$$

where s is the size of a d-dimensional vector v. This means that larger elements are far more significant than smaller ones. While a true Euclidean distance would calculate the square root of this result, this is not necessary and is an additional computational cost, as the square root function is monotonic, therefore it does not change the order of items and is ignored here. It would be more accurate to describe this as the Squared L_2 norm, however as it gives the same ordering, it will just be referred to as L_2 for simplicity. From the previous example, $[10,0,0]$ and $[0,10,0]$ would be of size 100, whereas $[5,0,5]$ would only be of size 50. In contrast to the L_1 norm, this treats a vector with more of its total sum of elements

across fewer dimensions as larger when compared to one which has its total split more evenly across more dimensions.

The L_∞ norm does not consider the size of each dimension, but only the most significant. This can be expressed as:

$$s = \max(|v_1|, \dots, |v_d|)$$

where s is the size of a d-dimensional vector v. This means that only the largest constraint is considered when placing an item, which can be suitable, particularly in cases where one constraint is a bottleneck. From the previous example, $[10,0,0]$ and $[0,10,0]$ would be given size 10, and $[5,0,5]$ would be given size 5. In many real world scenarios, including resource allocation, it is far more likely that there is a higher requirement for one constraint because it is uncommon that requirements of each constraint will be similar.

As well as these simpler norms, some more complex measures of size are implemented in an attempt to yield a tighter packing. This is particularly for bin-centric approaches to the problem. The most used measure for this is the dot-product score (Panigrahy, et al., 2011). This takes the dot-product of each item with the remaining capacity of the currently open bin. Whichever item maximises this is considered to be the most suitable, assuming it fits. This can be described as:

$$s = v \cdot c = \sum_{i=1}^{n=d} v_i c_i$$

where s is the size of a d-dimensional vector v on a bin with remaining capacity c. Because it is dependent on the capacity of the currently open bin, it may be more appropriate to think of it as suitability score, rather than a static measure of size. The benefit of using the dot product is that it essentially weights each item based on how empty each dimension of the current bin is. This gives a more dynamic way of choosing which item to select, and in many cases can result in a more tightly packed bin. From the previously used example, a bin with remaining capacity $c = [5,5,5]$ would result in each of the 3 items having sizes 50, whereas with capacity $c = [10,0,5]$, the vectors would have the following assigned sizes:

- $[10,0,0]$: 100
- $[5,0,5]$: 75
- $[0,10,0]$: 0

On top of these, some more experimental measures of size will be tested. The first of these is a sort that uses the product of all non-zero elements of a vector. This is defined as:

$$\prod_{i=1,v_i>0}^{n=d} v_i$$

The idea behind this is that an item with its L_1 size split across many elements, it is treated as considerably larger than one that has the majority of its size within less elements. This means that from the example case, $[10,0,0]$ and $[0,10,0]$ have a size of 10, but $[5,0,5]$ would have a size of 25. This means that items that require a bin with available space in lots of dimensions is fitted first. Another way of thinking about this is to consider a vector's size as its n-dimensional volume. On a dataset where $0 \leq v_i \leq 1$, this would cause the opposite of the intended outcome, and the size would be smaller with more dimensions, therefore data would need to be normalised to $v_i \geq 1$ before sorting. However, this is not a problem in this project, as the datasets are already made up of only positive integers.

The next is a multi-factor sort, which considers both the average size of each element within the vector, and the range of the values. It then weights these based on the average of each of these on the whole dataset. This is defined as:

$$s = \frac{V_{avg} \times v_{avg}}{V_{avg} + V_{range}} + \frac{V_{range} \times v_{range}}{V_{avg} + V_{range}}$$

where s is the size of a vector $v \in V$. $V_{avg}, V_{range}, v_{avg}$, and v_{range} can be defined as:

$$V_{avg} = \frac{\sum_{i=1}^{n} \left(\frac{\sum_{j=1}^{n=d} v_{i_j}}{d} \right)}{n}$$

$$V_{range} = \frac{\sum_{i=1}^{n} (\max(v_i) - \min(v_i))}{n}$$

$$v_{avg} = \frac{\sum_{i=1}^{n=d} v_i}{d}$$

$$v_{range} = \max(v) - \min(v)$$

By weighting this on the averages across the whole dataset, the average size (which has the same ordering result as L_1) is a more significant part of the size if there are lots of balanced vectors in the dataset, and the range is more significant if there are lots of unbalanced vectors. The dynamic nature of this measure may help to get a good result on a variety of different datasets. Assuming the example is a dataset of size $n = 3$, then $V_{avg} = \frac{4}{3}$, $V_{range} = \frac{25}{3}$, so $[10,0,0]$ and $[0,10,0]$ have size $\frac{766}{87} \approx 8.805$ and $[5,0,5]$ has size $\frac{391}{87} \approx 4.494$. The small dataset is not a great example but does still give a good idea of how this would work. As the result is almost always a decimal, and the implementation for this project will deal with integers, the weights will need to be scaled up by 10^x (x to be decided later) to allow for a more precise values of V_{avg} and V_{range} to be used instead of just truncating them to integers.

The final measure used for sorting will be a weighted version of the L_1 norm that scales each dimension based on their demand across the entire dataset. The weight for each dimension can be calculated as

$$w_d = \frac{\sum_{i=1}^{n} v_{i_d}}{\sum_{i=1}^{n} \left(\sum_{j=1}^{n=d} v_{i_j} \right)}$$

where w_d is the weight for dimension d, and vector $v \in V$. Size is then calculated:

$$s = \sum_{i=1}^{n=d} w_i v_i$$

This means that if the dataset is more constrained by one dimension, that dimension is weighted more in the calculation of each vector's size. In a case where every vector is constrained by the same dimension, this would act more like the L_∞ norm, but on a more balanced dataset, it would act more like the L_1 norm. This allows the ordering to be more similar to that of the norm more appropriate to the dataset. The key benefit expected from this is that it reduces the cost of outliers within a relatively uncontested dimension. If dimension 3 was very low on all except one vector, and the outlier was $[0,0,10]$, then it

would still be treated as a very small vector because it is more than likely going to fit in any bin. In the example, the weights are:

$$w_1 = 0.5, w_2 = 0.\dot{3}, w_3 = 0.1\dot{6}$$

Resulting in the sizes $[10,0,0] \rightarrow 5, [0,10,0] \rightarrow 3.\dot{3}, [5,0,5] \rightarrow 3.\dot{3}$. As with the multi-factor sort, the fractional weights will be scaled up to give a more precise result while working with integers in the implementation stage of this project.

The example used above is very small and can trivially be fitted into the optimal 2 bins regardless of which sort is used by a first fit or best fit heuristic, but it does help to highlight the way in which the sizes are determined. There are 2 possible packings that meet this optimal criterion – either $[5,0,5]$ or $[10,0,0]$ in a bin on its own. The packing that would be chosen by sorting with each of the measures can be useful to see what type of data it favours, and where each may be more suitable. The hypothesis for this is that the dot product (Panigrahy, et al., 2011) will most likely result in the best result in the majority of cases, but at a higher computational cost, as it needs to recalculate suitability each time an item is fitted. It would make sense for the weighted L_1 measure to outperform the L_∞ norm in cases where some dimensions are more contested than others, as it compensates for outliers but largely works in the same way. The product measure may excel in cases where bins can be densely packed, and the goal is to exhaust dimensions simultaneously.

6.1.2 Sorting Algorithm

The decision of which sorting algorithm to use in a real world implementation would likely be case dependent. The nature of the datasets would usually be more consistent, and known in advance, but since this project is using a variety of different classes of benchmark, and different heuristics, the decision is made easier. The logical choice for a sort algorithm for this is the merge sort, as it provides stable results in $O(n \log(n))$ time. This means that the algorithm and measure of size are being tested more accurately, and the sorting algorithm does not have as much impact on time. It also ensures that items keep a relative order, i.e., if 2 equal items are in the input file, they will not switch places. The downside to a merge sort is that it has a higher memory requirement, which may make it unsuitable for

datasets with millions of (or more) vectors. However, none of the benchmark datasets are this large, therefore the memory impact is negligible.

The plan for implementation is to have a merge sort that takes a vector of 2-dimensional vectors as input, and sorts based on the second dimension of each vector. This avoids the need to implement multiple variants of the merge sort, and instead calculate the "size" of each item, place that into a vector with the index of the item in the first dimension, and then use the output vector from the sort to place items from the original input based on their index.

6.1.3 Benchmark Data

As mentioned in the literature review, there are 4 sets of benchmark datasets, totalling a combined 27 classes. The 2-dimensional benchmarks are based on (Capara & Toth, 2001), and contains 10 classes. Class 9 will not be used, as it has different limits in each dimension. Class 2 from this is notable as it contains random values for each dimension $1 \leq x \leq 1000$, making it more random than the other classes. Class 1 contains small values $100 \leq x \leq 400$, Class 3 contains values $200 \leq x \leq 800$, with classes 4 and 5 containing smaller items $50 \leq x \leq 200$ and $25 \leq x \leq 100$ respectively. Each of the first 5 classes has bin size $[1000,1000]$, and both dimensions of vectors are generated randomly with the same limits. Classes 6 – 8 have bin size of $[150,150]$. In class 7 item weights are correlated, and in class 8 they are anti-correlated. Class 10 uses bins of size $[100,100]$ and the dataset is comprised of triplets of items which fill bins exactly, based on (Falkenauer, 1996). Therefore, in class 10, the optimal solution is $\frac{n}{3}$ where n is the number of items. The triplet benchmark can be used to give a definitive gap to the optimal solution when testing.

For higher dimensional datasets, the 3 benchmark sets from Vectorpack (Mommessin, et al., 2025) will be used. Each class within this has varying sized datasets, each with dimensions $d \in \{3,5,10\}$. The Panigrahy benchmark has 9 classes proposed by (Panigrahy, et al., 2011). The first 8 classes follow the same specification as the first 8 classes from the (Capara & Toth, 2001) benchmark, but with more dimensions. Class 9 is defined as throwing $2d$ balls into d bins, where d is the number of dimensions. This gives a random set of variables $X_1, \dots, X_d$ where X_i is the number of balls that fell into bin i. This gives an average 2 balls per bin, but causes a negative correlation, as some dimensions will end up with more, causing

others to have less. The size of each dimension is scaled according to X_i, with some added randomness to give a more realistic dataset.

The Triplet benchmark contains 2 classes, generating triplets of items that fit exactly into bins. There is one class based on (Capara & Toth, 2001), and one based on (Falkenauer, 1996).

The final benchmark is the newly introduced benchmark set from (Mommessin, et al., 2025), which defines 6 classes. The first contains d groups of items, items in group h have a large size in dimension h where $1 \leq h \leq d$, and small in all other dimensions. Each group has $\frac{n}{d}$ items, with the exception of the final group, which may require more to reach the target size of the dataset. Classes 2 and 3 are small in every dimension, and classes 4, 5, and 6 contain items that are either large in every dimension, or small in every dimension. With these benchmark datasets, there is enough variety in data to thoroughly test the effectiveness of each heuristic.

6.2 Heuristics

6.2.1 First Fit

One of the most straightforward heuristics is the first fit heuristic. It simply takes each item and places them into the first bin they fit in, opening a new one if an item does not fit into any bin. This runs extremely quickly, as it does not need to sort the items, and the only check is whether the item fits into a bin. However, this will usually result in a poor result as it is not making any attempt to place items in an optimal bin, just the first one it fits into.

6.2.2 Item-Centric FFD

The item-centric first fit decreasing heuristic is one of the best researched, and fundamental heuristics in both the standard and vector bin packing problems. Building on the first fit heuristic, it sorts every item into descending order before any items are placed, then places each item into the first bin that it fits in, opening a new bin if the current item does not fit into another. This results in a fast runtime, and better results than first fit, with the only additional computational cost being the sorting of items at the start. This can be used with all of the measures of size mentioned in 5.1.1, except the dot product measure. This

heuristic will be implemented using all the applicable measure of size to sort items, so that they can be compared.

6.2.3 Bin-Centric FFD

The bin-centric first fit decreasing heuristic follows largely the same idea as the item-centric one, except that it only opens a new bin once no item fits into the currently open one. This allows the use of the dot product measure, as items only need to be compared to one bin at each selection, rather than every bin. Because you are checking items until nothing fits into the bin, it will often result in a very slightly slower performance than the item-centric FFD. Without using a better way of assigning items, such as dot product, it will end up with an identical result as if you had used that same initial sorting order with the item-centric variation. As well as using the dot product, this will be implemented using the L_1 norm so that the time performance of the algorithm itself can be compared with the item-centric one.

6.2.4 Grouped by Most Significant

This is the first of the newly introduced heuristics as part of this project. The way the grouped heuristic works is to separate items into lists based on their largest element, sort each list into descending order based on that element, then bin-centrically place items, taking an item from the list corresponding to the most empty dimension of the current bin. The idea behind this is that by filling the most empty dimension with an item weighted towards that dimension, it will place unbalanced items into bins with other unbalanced items, producing a tighter packing. It also means that if a large gap is available in one dimension, it guarantees that if an item is able to utilise the space, it is not wasted by placing a smaller, more balanced item. The hypothesis is that this will be most effective in cases with unbalanced bins, such as Panigrahy Class 8 and 9, and Mommessin Class 4, 5 and 6, particularly with lower dimensional instances.

6.2.5 First N Items

This heuristic has been designed to prioritise time performance over a tight packing but still provide better results than a first fit algorithm. The way it works, is to sort items into descending order, and calculate a lower bound:

$$lb = \max_{i=1}^{n=d} \left(\sum_{j=1}^{n} v_{i_j} \right)$$

Then place the lb largest items into separate bins, then place the remaining items using an item-centric FFD algorithm. While this may not provide the best results when compared to other heuristics, but by placing the first lb items, it takes slightly more computation time than a FFD on a dataset of $n - lb$ items. The expectation with this is that it will perform the fastest on a dataset of mostly large items because less items need to be fitted by the FFD, but it is expected produce its most optimal results on datasets with ~ 3 items per bin, and similarly sized items, as it finds a good balance between time performance and a good result. Based on this, it is expected that this would give the best on the triplet instances and give a faster result than FFD on all datasets.

6.2.6 Decaying Lookahead

This heuristic is the only non-greedy heuristic to be implemented as part of this project. The idea is fundamentally similar to the standard FFD algorithm but uses a recursive function to test multiple packings before closing a bin. The main focus of this heuristic is to give a very good result in the majority of instances. It follows these steps:

1) Calculate a lower bound, and then an estimate per bin $est = \frac{\text{size}(V)}{lb}$.
2) Calculate a starting value of lookahead $l = \max\left(z,\ ceil\left(\frac{x}{est}\right)\right)$ and decay $d = \frac{y}{est}$.
   Unlike most variables in this project, decay is a float rather than an integer, to give finer control over the decay of the lookahead. The x, y & z variables will be chosen from testing, as they limit the maximum size of both the initial lookahead, and decay. They should create a balance between mostly consistent time, and an optimal result.
3) Sort the dataset into descending order using the L_1 norm.
4) Until there are no remaining vectors ($V = \emptyset$), fill bins:
   a. Open a bin and initialise its capacities.
   b. For $1 \rightarrow l$:
      i. Find the first untested item that fits, provisionally place it, and call the placement function again with $l = \max(1, l - d)$ to place the next item.

ii. If no item fits in the bin, then the stop condition is reached. Choose the tightest placement from each tested item and go up one level of recursion.

c. Once the bin has been filled, return the tightest packing from all the tested packings, and repeat for the next bin.

A simple example of packing a bin:

1) $V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}, l = 3, d = 1$. For this example, assume any 3, but no 4, items $v_i \in V$ fit into a bin together.
2) Bin is opened, initially fitted with v_1, then v_2, then v_3
3) At the deepest level, $l = l - c - c = 3 - 1 - 1 = 1$, so only v_3 is tested.
4) Once it backtracks to v_1, v_2, $l = l - c = 3 - 1 = 2$, so the packing is tested again with v_1, v_3, v_4.
5) This then backtracks to the top level where $l = 3$, and v_1 is ignored for the next recursions. The steps are the same as above, so all tested cases are:
   a. v_2, v_3, v_4
   b. v_2, v_4, v_5
   c. v_3, v_4, v_5
   d. v_3, v_5, v_6
6) The best of these 6 packings is chosen, and those items are removed from V.

The maximum number of cases – fewer if V runs out of items – tested c can be expressed as:

$$c = \prod_{i=0}^{n=\left(\text{floor}\left(\frac{l-1}{d}\right)\right)} \text{ceil}(l - id)$$

Therefore if $d = 0$, $c = \infty$, and if $d \geq l$, $c = l$. The hypothesis for the decaying lookahead heuristic is that it will outperform all the other tested algorithms with its result but take the longest time. It can be expected that it will occasionally take a long time, if there is a large number of bins to be filled, the initial lookahead is high, and the decay is low. The closer the result is to the lower bound should, as an estimate, result in a faster runtime than if the result is much higher than the lower bound, as the lookahead and decay are calculated based on the lower bound and number of items, the lower bound is the only one of these

two values that is not guaranteed to be accurate, i.e., the number of items is known, but the optimal solution is not.

7. Implementation

7.1 Framework

In this section, any code that is not one of the heuristics will be drafted and explained.

7.1.1 main.cpp

The main function is where the input data will be prepared, and the results will be processed and output. Initially, this was programmed to run one dataset at a time so that heuristics could be tested during development, but it has since been updated to run multiple datasets from a specified directory.

```
Import heuristics
// array of heuristics to be used
heuristics = {1,2,3}

// constant to determine which format of dataset is used
// not the most robust way to do this, but since this code is only
// for testing heuristics, it is the simplest & therefore has been chosen.
const bool 2CBPDataset = true/false

function main():
	string inputFolderPath = "./datasetClass/"

	// initialise variables to store cumulative results
	int datasetsUsed = 0
	list<int> totalTime = {}
	list<int> totalBins = {}
	list<int> totalConfirmedOptimal = {}
	int totalLowerBound = 0

	firstRun = true
	for (inputFilePath in inputFolderPath)
		// regex to select certain files (only 3 dimensional datasets
		// with 60 items)
		// this is not the most robust implementation, but allows for
		// quick testing and lots of control over benchmarks used
		if (not regex.match(inputFilePath, "*_60_3_*"))
```

```
                continue
            datasetsUsed += 1
            inputFile = inputFilePath.open()
            // get bin size from start of dataset file
            binSize = getBinSize(inputFile.line())
            // get dataset as list of vectors
            list<vector> dataset = getDatasetArray(inputFile, 2CBPDataset)
            lowerBound = getLowerbound(dataset, binSize)
            totalLowerBound += lowerBound
            for (i in heuristics.size())
                  // get current heuristic, and start time
                  int heuristic = heuristics[i]
                  int start = date.now().microseconds()
                  // run heuristics, storing result
                  result = runHeuristic(heuristic, dataset, binSize)
                  // get end time, and calculate duration in microseconds
                  int stop = date.now().microseconds()
                  int duration = stop - start
                  // add results to cumulative totals
                  if (firstRun)
                        totalBins.add(result.size())
                        totalTime.add(duration)
                        if (result.size() == lowerBound)
                              totalConfirmedOptimal.add(1)
                        else
                              totalConfirmedOptimal.add(0)
                  else
                        totalBins[i] += result.size()
                        totalTime[i] += duration
                        if (result.size() == lowerBound)
                              totalConfirmedOptimal[i] += 1
            ensure firstRun is no longer true
            if (firstRun)
                  firstRun = false
      print("Tested on " & datasetsUsed & " datasets")
      print("Average lower bound: " & totalLowerBound / datasetsUsed)
      for (i in heuristics.size())
            print("Heuristic: " & heuristics[i])
            print("Mean time: " & totalTime[i] / datasetsUsed & " - Mean
bins: " & totalBins[i] / datasetsUsed)
            if (totalConfirmedOptimal[i] > 0)
                  print(totalConfirmedOptimal[i] & " confirmed optimal
results")

int function getBinSize(inputString)
      // bool to check for leading spaces
```

```
	bool padding = true
	string binSize = “”
	for (i in inputString.size())
		// add non-space characters to size until next space reached
		if (padding && inputString[i] == ‘ ‘)
			continue
		if (padding)
			padding = false
			binSize += inputString[i]
		elseif (inputString[i] == ‘’)
			return (toInt(binSize))
		else
			binSize += inputString[i]
	return (toInt(binSize))

int function getLowerBound(dataset, binSize)
	// calculate minimum bins required for each dimension
	int binsReqd = 0
	for (i in dataset[0].size() - 1)
		int total = 0
		for (item in dataset)
			total += item[i]
		// check if lower bound for dimension is higher than current
		// lower bound
		if (binsReqd < ceil(total/binSize))
			binsReqd = ceil(total/binSize
	// return lower bound
	return binsReqd

list<vector> function getDatasetArray(inputFile, 2cbp)
	// initialise list and counter
	list<vector> dataset = {}
	int counter = 0
	do until (inputFile.EOF())
		// add vectors to array format (index, v_1,...,v_d)
		if (2cbp)
			dataset.add(parse2CBPLine(counter, inputFile.line())
		else
			dataset.add(parseLine(counter, inputFile.line())
		counter += 1
	return (dataset)
```

7.1.2 mergeSort.cpp

The sorting algorithm plays a crucial role in the majority of the implemented heuristics. For the variations of the item-centric FFD heuristics, it is the only thing that differentiates them

(L_1, L_2, etc.). Below is the pseudocode for the sort, and the calculation of size for the different measures of size.

```
// input list should contain vectors {index, size}
list<vector> function mergeSort(dataset)
	// if dataset is smaller than 2 items, it is already sorted
	if (dataset.size() <= 1)
		return dataset
	int middle = floor(dataset.size() / 2)
	// split list, and pass halves into recursive function
	list<vector> left = mergeSort(dataset[0..middle])
	list<vector> right = mergeSort(dataset[(middle + 1)..(dataset.size()
- 1)])

	return (merge(left, right))

list<vector> function merge(left, right)
	// index both lists, and result
	int ileft,iright,i = 0
	int lsize = left.size()
	int rsize = right.size()
	list<vector> result[lsize+rsize]

	// compare items until list is empty
	While (ileft < lsize && iright < rsize)
		if (left[ileft][1] < right[iright][1])
			result[i] = right[iright]
			i += 1
			iright += 1
		else
			result[i] = left[ileft]
			i += 1
			ileft += 1

	// append remaining values to end then return
	While (ileft < lsize)
		result[i] = left[ileft]
		i += 1
		ileft += 1
	while (iright < rsize)
		result[i] = right[iright]
		i += 1
		iright += 1
	return result

list<vector> function sortL1(dataset)
```

```
	list<vector> sortInput[dataset.size]
	list<vector> sortedDataset = dataset
	// get L1 size of each vector
	for (item in dataset)
		bool first = true
		int sum = 0
		for (element in item)
			if (first)
				first = false
			else
				sum += element
		sortInput[item[0]] = {item [0], sum}
	list<vector> sortResult = mergeSort(sortInput)
	// insert dataset into sortedDataset in the sorted order
	for (i in sortResult.size())
		sortedDataset[i] = dataset[sortResult[i][0]
	return sortedDataset

list<vector> function sortL2(dataset)
	list<vector> sortInput[dataset.size]
	list<vector> sortedDataset = dataset
	// get L2 size of each vector
	for (item in dataset)
		bool first = true
		int sum = 0
		for (element in item)
			if (first)
				first = false
			else
				sum += element^2
		sortInput[item[0]] = {item[0], sum}
	list<vector> sortResult = mergeSort(sortInput)
	// insert dataset into sortedDataset in the sorted order
	for (i in sortResult.size())
		sortedDataset[i] = dataset[sortResult[i][0]
	return sortedDataset

list<vector> function sortLinf(dataset)
	list<vector> sortInput[dataset.size]
	list<vector> sortedDataset = dataset
	// get L-infinity size of each vector
	for (item in dataset)
		bool first = true
		int size = 0
		for (element in item)
			if (first)
```

```
                        first = false
                  elseif (element > size)
                        size = element
            sortInput[item[0]] = {item[0], size}
      list<vector> sortResult = mergeSort(sortInput)
      // insert dataset into sortedDataset in the sorted order
      for (i in sortResult.size())
            sortedDataset[i] = dataset[sortResult[i][0]
      return sortedDataset

list<vector> function sortByProduct(dataset)
      list<vector> sortInput[dataset.size]
      list<vector> sortedDataset = dataset
      // get volume of each vector
      for (item in dataset)
            bool first = true
            int product = 1
            for (element in item)
                  if (first)
                        first = false
                  elseif (element > 0)
                        product *= element
            sortInput[item[0]] = {item[0], product}
      list<vector> sortResult = mergeSort(sortInput)
      // insert dataset into sortedDataset in the sorted order
      for (i in sortResult.size())
            sortedDataset[i] = dataset[sortResult[i][0]
      return sortedDataset

list<vector> function sortL1Weighted(dataset)
      list<vector> sortInput[dataset.size]
      list<vector> sortedDataset = dataset
      int totalSum = 0
      list<int> sums
      // calculate sum of all elements, and all elements for each
      // dimension
      for (i in dataset[0].size() - 1)
            sums.add(0)
      for (item in dataset)
            for (i=1 in item.size())
                  sums[i - 1] += item[i]
                  totalSum += item[i]

      // calculate weights
      list<int> weights
      for (i in sums.size() - 1)
```

```
            weights.add(toInt(100000.0 * sums[i] / totalSum))
      // get L1 size of each vector with weighted values
      for (item in dataset)
            bool first = true
            int sum = 0
            for (i in item.size())
                  if (first)
                        first = false
                  else
                        sum += item[i] * weights[i - 1]
            sortInput[item[0]] = {item[0], sum}
      list<vector> sortResult = mergeSort(sortInput)
      // insert dataset into sortedDataset in the sorted order
      for (i in sortResult.size())
            sortedDataset[i] = dataset[sortResult[i][0]
      return sortedDataset

list<vector> function multiFactorSort(dataset)
      list<vector> sortInput[dataset.size()]
      list<vector> sortedDataset = dataset;
      list<vector> vectorRangeAndProduct[dataset.size(), 3]
      int avgRange = 0
      int avgAvg = 0
      // get the difference between largest and smallest dimension
      // get average size of vector elements
      for (item in dataset)
            int largest = 1
            int smallest = 1
            int avg = 1
            for (i=1 in item.size())
                  avg += item[i]
                  if (item[i] < item[smallest])
                        smallest = i
                  else if (item[i] > item[largest])
                        largest = i
            avg /= (item.size() - 1)
            int range = item[largest] - item[smallest]
            avgRange += range
            avgAvg += avg
            vectorRangeAndProduct[item[0]] = { item[0], range, avg }
      avgRange /= dataset.size()
      avgAvg /= dataset.size()
      int rangeWeight = round(1000.0 * avgRange / (avgRange + avgAvg))
      int avgWeight = 1000 - rangeWeight
      for (i in dataset.size() - 1)
            sortInput[i][0] = vectorRangeAndProduct[i][0]
```

```
            int size = vectorRangeAndProduct[i][1] * rangeWeight +
vectorRangeAndProduct[i][2] * avgWeight
            sortInput[i][1] = size
      // order dataset based on sort result
      list<vector> sortResult = mergeSort(sortInput)
      for (i in sortResult.size() -1)
            sortedDataset[i] = dataset[sortResult[i][0]]
      return sortedDataset
```

7.2 Heuristics

For each heuristic, pseudocode will be given to ensure they can be easily understood.

7.2.1 Item-Centric First Fit (Decreasing)

Since the logic is identical for First Fit, and FFD, except for the dataset needing to be sorted, and there is already pseudocode for the sort algorithms. Because of this, it can be assumed that for FFD, it uses the same code as First Fit, with a pre-sorted dataset passed into the function.

```
list<vector> function firstFit(dataset, binSize)
      list<vector> bins = {{}}
      list<vector> binCapacities
      vector newBin
      // initialise first bin
      for (i in dataset[0].size())
            newBin.add(binSize)
      binCapacities.add(newBin)
      // place each item
      for (item in dataset)
            bool fit = false
            bin = 0
            while (!fit)
                  if (bin >= bins.size())
                        binCapacities.add(newBin)
                        bins.add({item[0]})
                        fit = true
                  else
                        fit = true
                        for (i in item.size() - 1)
                              if (item[i + 1] > binCapacities[bin][i])
                                    fit = false
                                    break
                        if (fit)
                              binCapacities[bin] -= item[1, item.size()-1]
```

```
                        bins[bin].add(item[0])
                bin += 1
        return bins
```

7.2.2 Bin-Centric FFD

While this heuristic gives the same result as the item-centric FFD, it is useful for comparison of time performance. As with the item-centric pseudocode, it can be assumed that the list is already sorted.

```
list<vector> function FFDBinCentric(dataset, binSize)
        itemSize = dataset[0].size() - 1
        // initialise bins and bin capacity variables
        list<vector> bins = {}
        list<vector> remainingCapacities
        vector newBin
        for (i in itemSize - 1)
                newBin.add(binSize)
        int currentBin = -1
        // while there are items in the dataset, open new bin and fill it
        while (dataset.size() > 0)
                // add the bin
                currentBin += 1
                bins.add({})
                remainingCapacities.add(newBin)
                // iterate through dataset, checking if item fits
                int currentItem = 0
                while (currentItem < dataset.size())
                        bool fit = true
                        for (i=1 in dataset[currentItem].size())
                                if (dataset[currentItem][i] >
remainingCapacities[currentBin][i - 1]
                                        fit = false
                                        currentItem += 1
                                        break
                        // if item fits, place it in bin
                        if (fit)
                                bool full = true
                                bins[currentBin].add(dataset[currentItem[0]])
                                for (i in itemSize - 1)
                                        remainingCapacities[currentBin][i] -=
dataset[currentItem][i + 1]
                                        if (remainingCapacities[currentBin][i] > 0)
                                                full = false
                                dataset.remove(currentItem)
                                if (full)
```

```
                        break
        return bins
```

7.2.3 Dot-Product Bin-Centric FFD (Panigrahy, et al., 2011)

```
list<vector> function PanigrahyDotProduct(dataset, binSize)
        itemSize = dataset[0].size() - 1
        list<vector> bins = {}
        list<vector> remainingCapacities
        vector newBin
        for (i in itemSize – 1)
                newBin.add(binSize)
        int currentBin = -1
        // while dataset contains items, open bin and fill it
        while (dataset.size() > 0)
                // open new bin
                currentBin += 1
                bins.add({})
                remainingCapacities.add(newBin)
                itemFits = true
                // while an item fits in the bin, keep placing items
                while (itemFits)
                        // break if dataset is empty
                        if (dataset.size() == 0)
                                break
                        // get the best fitting item for the current capacity
                        int nextItem =
getBestFit(remainingCapacities[currentBin], dataset, itemSize)
                        // if nextItem = -1, no item fits
                        if (nextItem == -1)
                                itemFits = false
                        else
                                // place best fitting item
                                bins[currentBin].add(dataset[nextItem][0])
                                for (i in itemSize - 1)
                                        remainingCapacities[currentBin][i] -=
dataset[nextItem][i + 1]
                                dataset.remove(nextItem)
        return bins


int getBestFit(capacity, dataset, itemSize)
        int best = 0
        int bestItem = -1
        // iterate through dataset, getting best fitting item's index
        for (i in dataset.size() - 1)
                bool fits = true
```

```
            for (j in itemSize - 1)
                  if (dataset[i][j + 1] > capacity[j])
                        fits = false
                        break
            if (fits)
                  int dotProduct = getDotProduct(dataset[i], capacity,
itemSize
                  if (dotProduct > best)
                        best = dotProduct
                        bestItem = i
      return bestItem

int getDotProduct(item, capacity, itemSize)
      // calculate and return the dot product of the item and
      // remaining capacity
      int total = 0
      for (i in itemSize - 1)
            total += item[i + 1] * capacity[i]
      return total
```

7.2.4 Grouped By Most Significant

This is the first of the newly introduced heuristics.

```
list<vector> function GroupedByMostSignificant(dataset, binSize)
      int itemSize = dataset[0].size()
      list<vector> fullDataset = dataset
      list<list<vector>> splitDataset
      list<vector> bins
      list<vector> remainingCapacities
      vector newBin
      for (i in itemSize – 1)
            newBin.add(binSize)
      for (i=1 to itemSize - 1)
            splitDataset.add ({})
            // go through each remaining item
            for (j in dataset.size() - 1)
                  vector item = dataset[j]
                  // check if element is the most significant value
                  bool largest = true
                  for (k=1 in item.size() - 1; k++)
                        if (item[k] > item[i])
                              largest = false
                  // add to split dataset, remove from dataset, decrement
j to compensate for removal
                  if (largest)
                        splitDataset[i - 1].add(item)
```

```
                        dataset.remove (j)
                        j -= 1
      // sort each list by its most significant value
      for (i in splitDataset.size() - 1)
            list<vector> listToSort
            for (item in splitDataset[i])
                  listToSort.add({ item[0], item[i + 1] })
            listToSort = sort(listToSort)
            list<vector> sortedList
            for (item in listToSort)
                  sortedList.add(fullDataset[item [0]])
            splitDataset[i] = sortedList
      bool datasetEmpty = false
      int currentBin = -1
      while (!datasetEmpty)
            bins.add({})
            currentBin += 1
            remainingCapacities.add(newBin)
            bool binFull = false
            // vector of lists in splitDataset that may have an item that
fits in bin
            vector untestedLists
            for (i in splitDataset.size() - 1)
                  untestedLists.add(i)
            while (!binFull)
                  // get list of the most empty dimension(s)
                  vector mostRemainingSpace = {untestedLists[0]}
                  for (i in untestedLists)
                        if (remainingCapacities[currentBin][i] >
remainingCapacities[currentBin][mostRemainingSpace[0]])
                              mostRemainingSpace = { i }
                        elseif (remainingCapacities[currentBin][i] ==
remainingCapacities[currentBin][mostRemainingSpace[0]])
                              mostRemainingSpace.add(i)
                  int listToCheck = mostRemainingSpace[0]
                  // check which list has the most items left
                  if (mostRemainingSpace.size() > 1)
                        for (i in mostRemainingSpace)
                              if (splitDataset[listToCheck].size() <
splitDataset[i].size())
                                    listToCheck = i
                  bool addedItem = false
                  for (i in splitDataset[listToCheck].size())
                        vector item = splitDataset[listToCheck][i]
                        bool fit = true
                        for (j=1 in itemSize)
```

```
                              if (item[j] >
remainingCapacities[currentBin][j - 1])
                                    fit = false
                                    break
                        if (fit)
                              addedItem = true
                              bins[currentBin].add(item[0])
                              for (j=1 in itemSize - 1)
                                    remainingCapacities[currentBin][j - 1]
-= item[j]
                              splitDataset[listToCheck].remove(i)
                              break
                  if (!addedItem)
                        for (i in untestedLists.size() - 1)
                              if (untestedLists[i] == listToCheck)
                                    untestedLists.remove(i)
                  if (untestedLists.size() == 0)
                        binFull = true
            // check if all lists are empty before looping
            datasetEmpty = true
            for (list in splitDataset)
                  if (list.size() > 0)
                        datasetEmpty = false
```

7.2.5 First N Items

The sort algorithm for this will likely be using the $L1$ norm, but in the pseudocode, it will just use a function “sort” to show that this is a flexible choice depending on performance of other sort methods tested.

```
list<vector> function FirstNItems(dataset, binSize)
      dataset = sort(dataset)
      int lowerBound = 0
      for (i=1 in dataset.size() – 1)
            float total = 0
            for (item in dataset)
                  total += item[i]
            if (total / binSize > lowerBound)
                  lowerBound = ceil(total / binSize)
      list<vector> bins
      list<vector> remainingCapacities
      vector newBin
      for (i in itemSize – 1)
            newBin.add(binSize)
      for (i in lowerBound)
            bins.add({dataset[i][0]})
```

```
            remainingCapacities.add(newBin - (dataset[i][1,
dataset[i].size() - 1]))
      for (i in lowerBound - 1 to 0)
            dataset.remove(dataset[i])
      for (item in dataset)
            bool fit = false
            int bin = 0
            while (!fit)
                  if (bin < bins.size())
                        fit = true
                        for (i=1 in item.size() - 1)
                              if (item[i] > remainingCapacitites[bin][i-1]
                                    fit = false
                                    break
                        if (fit)
                              bins[bin].add(item[0])
                              remainingCapacities[bin] -= item[1,
item.size()-1]
                  else
                        // create new bin
                        bins.add({item[0]})
                        remainingCapacities.add(newBin - item[1,
item.size()-1]
                        fit = true
                  bin += 1
      return bins
```

7.2.6 Decaying Lookahead

```
list<vector> function DecayingLookahead(dataset, binSize, lowerBound)
      int datasetSize = dataset.size()
      int vectorSize = dataset[0].size()
      float estimatePerBin = dataset.size() / lowerBound
      int initialLookforward = max(3, ceil(25/estimatePerBin))
      float decayRate = 4/estimatePerBin
      dataset = sort(dataset)
      list<vector> bins
      while (dataset.size() > 0)
            bins.add(fillBin(dataset, binSize, initialLookforward,
decayRate)
      return bins

vector function fillBin(byRef dataset, binSize, initialLookforward,
decayRate)
      vector bin = {dataset[0][0]}
      vector capacity
      for (i in dataset[0].size() - 1)
```

```
            capacity.add(binSize - dataset[0][i + 1])
      vector fullBin = placeItems(dataset, bin, capacity,
float(initalLookforward), decayRate, 1)
      // remove placed items
      int datasetIndex = 0
      vector toRemove
      for (i in fullBin.size() - 1)
            for (j in datasetIndex to dataset.size()
                  if (dataset[j][0] == fullBin[i])
                        datasetIndex += (j + 1)
                        toRemove.add(j)
                        break
      for (i in  toRemove.size() - 1 to 0)
            dataset.remove(i)
      return fullBin

vector function placeItems(byRef dataset, bin, byref capacity, lookahead,
decayRate, startItem)
      // if no items remain, stop condition is reached
      if (startItem >= dataset.size())
            return bin
      // reduce lookahead
      float newLookahead = lookahead - decayRate
      if (newLookahead < 1)
            newLookahead = 1
      vector newBin = bin
      vector newCapacity = capacity
      // test l items (where l = lookahead)
      for (i in ceil(lookahead) - 1)
            vector testBin = bin
            bool itemFit = false
            for (j=startItem in dataset.size() - 1)
                  if (checkItem(capacity, dataset[j]))
                        for (k=1 in dataset[j].size())
                              capacity[k - 1] -= dataset[j][k]
                              testBin.add (dataset[j][0])
                              testBin = placeItems(dataset, testBin,
testCapacity, newLookahead, decayRate, (j + 1))
                              itemFit = true
                              startItem = j + 1
                              break
            if (!itemFit) {
                  break
            int bestTotal, newTotal = 0
            for (j in dataset[0].size() - 1)
                  bestTotal += newCapacity[j]
```

```
                  newTotal += testCapacity[j]
            if (newTotal < bestTotal)
                  newBin = testBin
            else
                  capacity = newCapacity
      return newBin

bool checkItem(capacity, item) {
      int fit = true
      // check if any elements don't fit in the bin
      for (i=1 in item.size() - 1)
            if (item[i] > capacity[i - 1])
                  fit = false
                  break
      return fit
```

8. Evaluation of Implemented Heuristics

8.1 Sorting Methods

The results for each measure of size on each benchmark will be evaluated below. This will be shown as an average number of bins used for instances of the same number of dimensions and items.

8.1.1 2CBP

This benchmark differs from the others, as it is the only 2-dimensional benchmark. This is a relevant benchmark for cases of simple VM allocation with just memory and CPU requirements or loading shipping containers with weight and volume constraints. For each class, there are 10 instances of each dataset size.

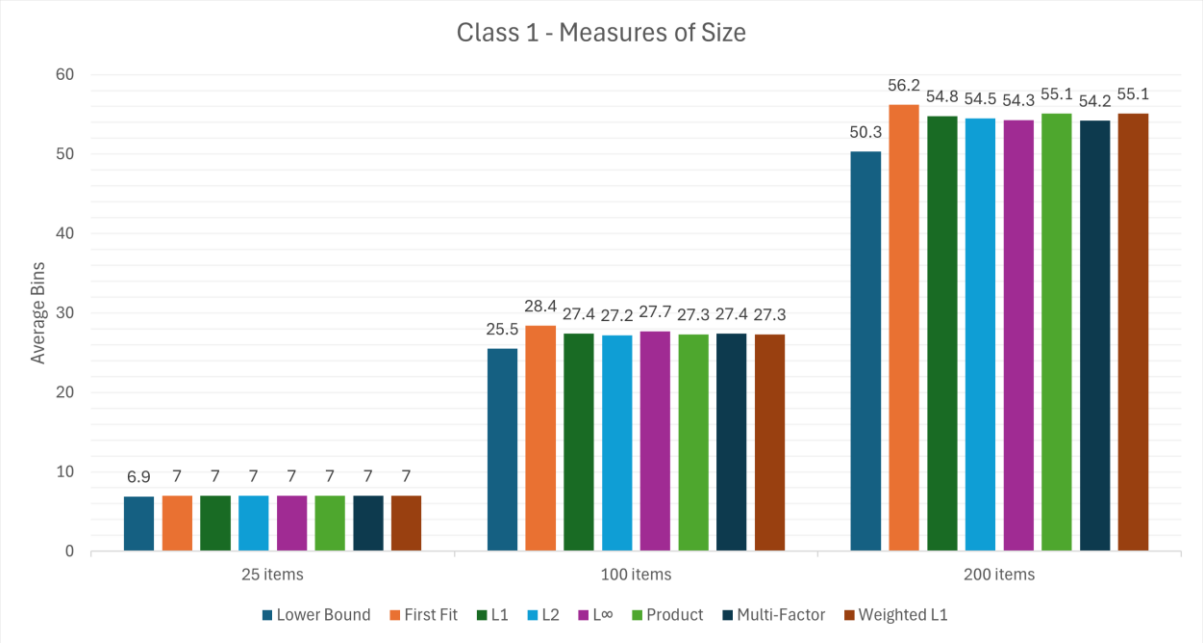


Figure 1 - Comparison of Measures of Size for 2CBP Class 1

As seen in *Figure* 1 above, on the 25 item datasets, all sorts, including no sort, gave identical results. Each gave definitely optimal ($result = lb$) results on 9 of the 10 datasets. On the medium sized input, the L2 norm gave the best result, and L∞ performed the worst, but by a small margin (0.4 bin range between results). They all performed much better than the unsorted first-fit heuristic – at least 0.7 bins better for 100 items, and 1.1 for 200 items – demonstrating that the suggested sorting methods have at least some level of effectiveness.

The multi-factor sort performed 0.2 bins worse than the best result on 100 items, and the best on 200 items, suggesting that it is suited well to 2-dimensional cases, and scales well. This makes sense as the nature of the dataset may begin to play a more important role in finding a good packing as the number of items increases, because the scale begins to reduce the impact of outliers.

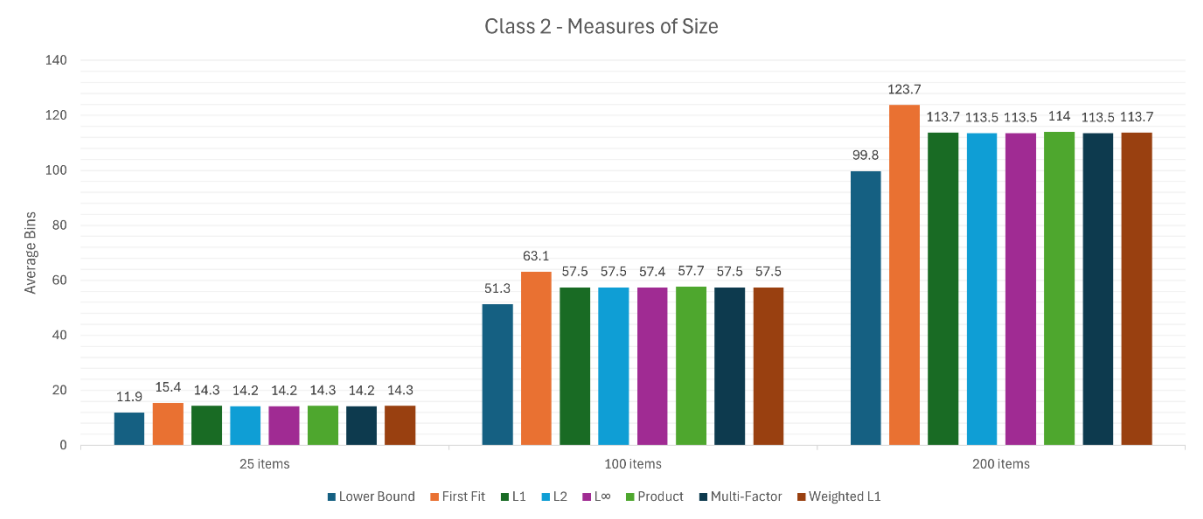


Figure 2 - Comparison of Measures of Size for 2CBP Class 2

Class 2 gives a dataset of random items from 1 to 1000, on bins of size 1000 in each dimension. As seen in *Figure 2*, this does not favour any of the sorts, with a range of no more than 0.5 bins across all dataset sizes, but each outperforming the unsorted first fit by at least 1 bin, and over 5 bins in the 100 and 200 item benchmarks. This result definitively shows that all suggested measures of size can be suitable and effective in some cases, as they all perform similarly on a random dataset, and better than first fit. The other classes will be useful to determine what type of dataset each is more suited to.

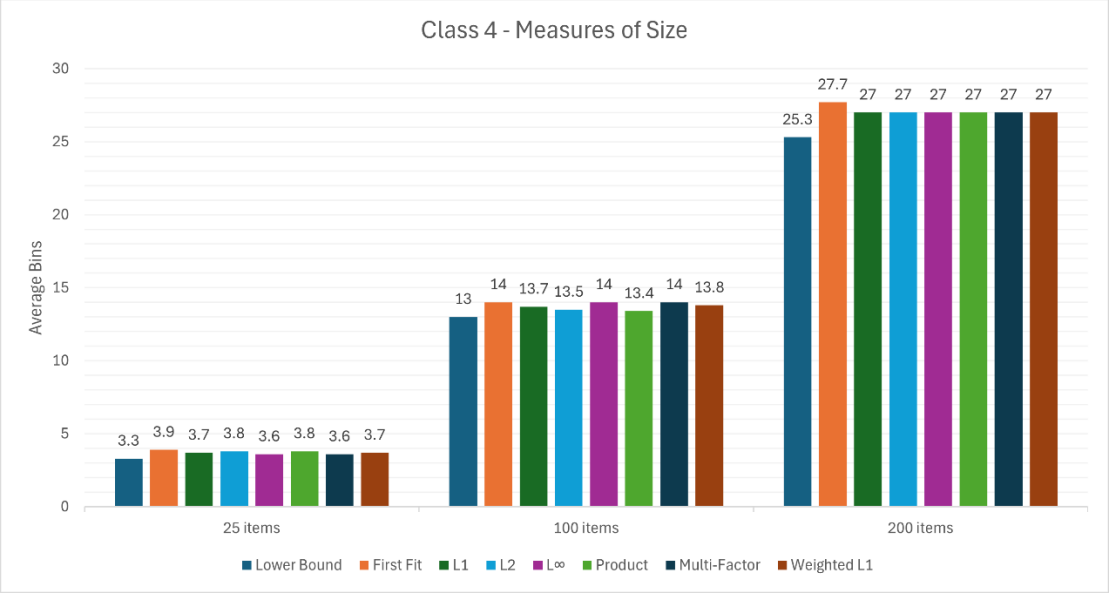


Figure 3 - Comparison of Measures of Size for 2CBP Class 4

Figure 3 shows that for Class 4, which is made up of very small items, the performance is negligibly better than that of the first fit algorithm on smaller datasets. Once the dataset is larger, the difference in performance across sorting algorithms is less than 1 bin. For this reason, it is logical that if the dataset is comprised of very small, low dimensional items, and time complexity is important, it is not unreasonable to use the first-fit heuristic, and the sorting method chosen for a FFD doesn't greatly influence performance.

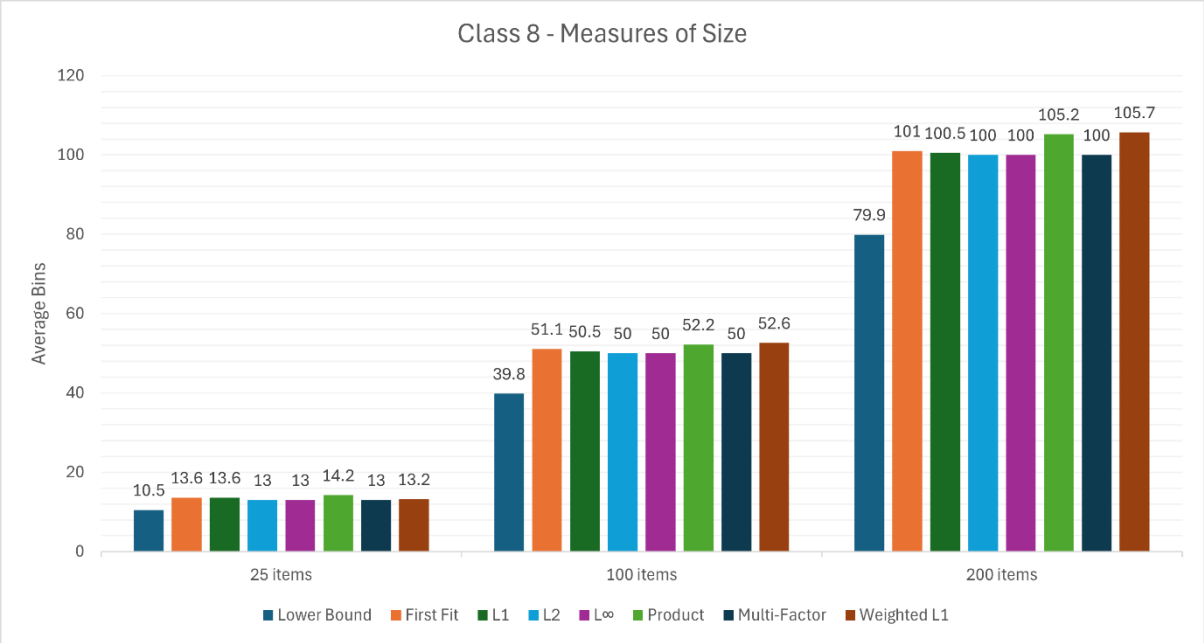


Figure 4 - Comparison of Measures of Size for 2CBP Class 8

Class 8 contains negatively correlated items and reveals a flaw in some of the newly proposed sorting methods. *Figure 4* shows that the product and weighted L1 sorts are outperformed by the first-fit algorithm, and the poor quality of the results increases with the size of the dataset. The others all perform marginally better than the first fit heuristic, and within 0.6 bins of each other. For this reason, if the nature of the dataset is not certain, the product and weighted L1 sorts should be avoided. Unless these have consistently good performance on higher dimensional instances, it is unlikely that they will be a suitable choice for any cases.

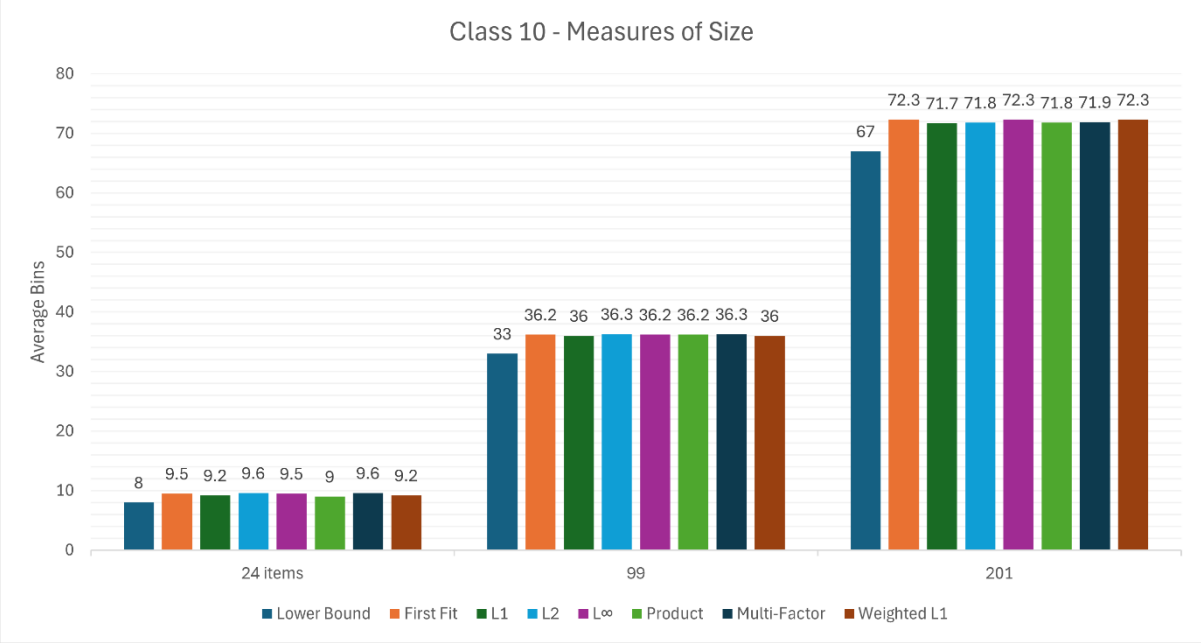


Figure 5 - Comparison of Measures of Size for 2CBP Class 10

Since Class 10 contains a possible perfect fit for every item, the lower bounds are guaranteed to be the optimal solution, and equal for each instance of the same size – hence the slightly altered dataset sizes. From *Figure 5*, we can see that performance is very similar across all the proposed sorting methods, and most of them marginally outperform first fit.

Across all of the tested classes, the L1 and L2 sorts give the most consistent results, however some others give a better result in specific cases. The multi-factor sort is the only one of the three newly proposed sorts that consistently gives a good result, as the other two perform poorly on negatively correlated datasets. The overall conclusion from this set of benchmark classes is that the sorting algorithm chosen for a 2-dimensional instance does not greatly

influence the performance, and a simple measure such as L1 or L2 is likely to be the most suitable choice due to their stable and good results.

8.1.2 Panigrahy

The Panigrahy benchmark are based on (Panigrahy, et al., 2011) and contain 3, 5 and 10 dimensional instances, with similar methods of generation to the 2CBP benchmark.

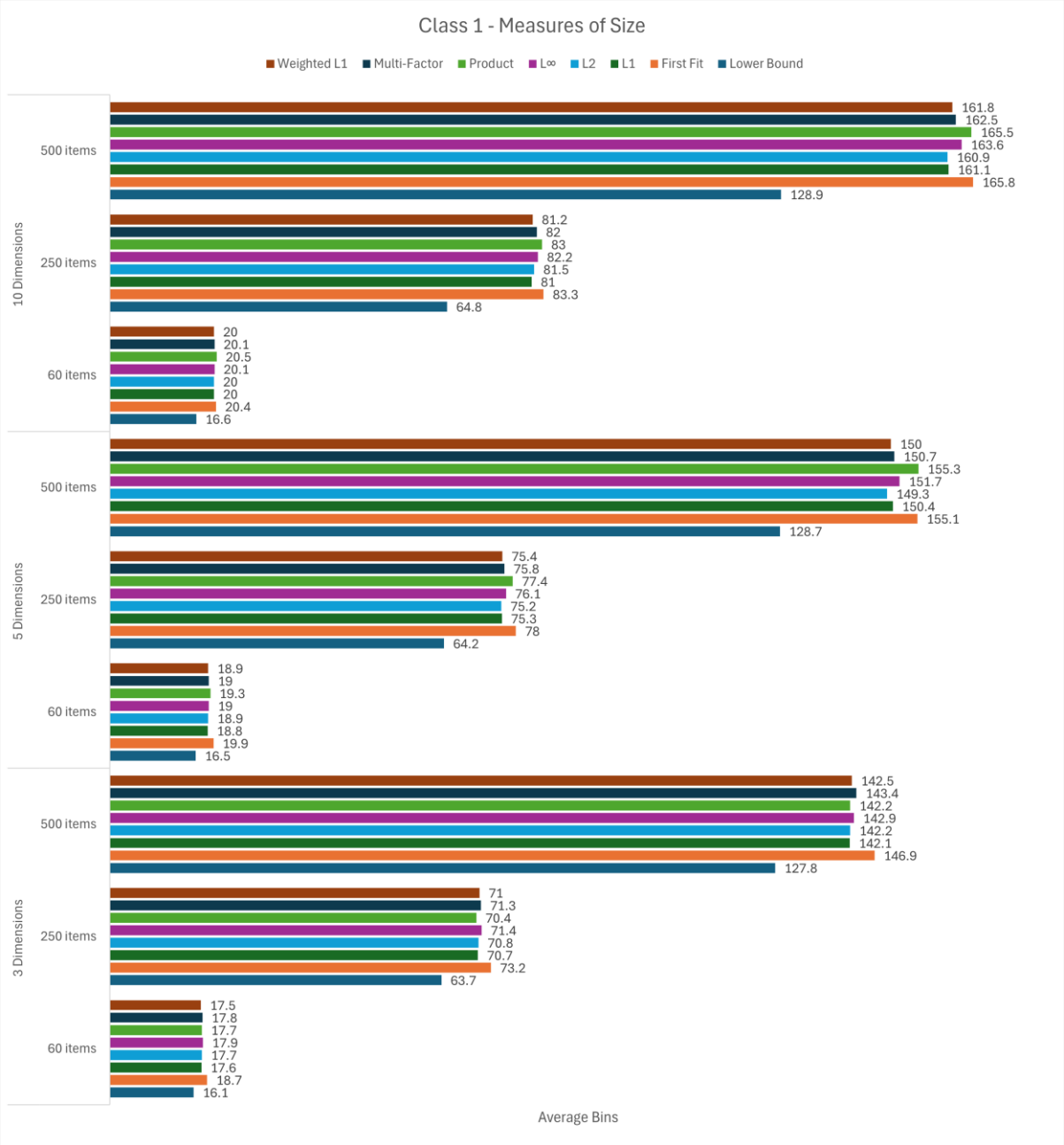


Figure 6 - Comparison of Measures of Size for Panigrahy Class 1

When comparing the results in *Figure 6* and *Figure 1*, we can see that most notably, the product sort scales particularly poorly to higher dimensions and larger datasets, with it performing worse than first fit in many 5 and 10 dimensional instances. L1 and L2 continue to give consistently positive results even on large datasets with high dimensions.

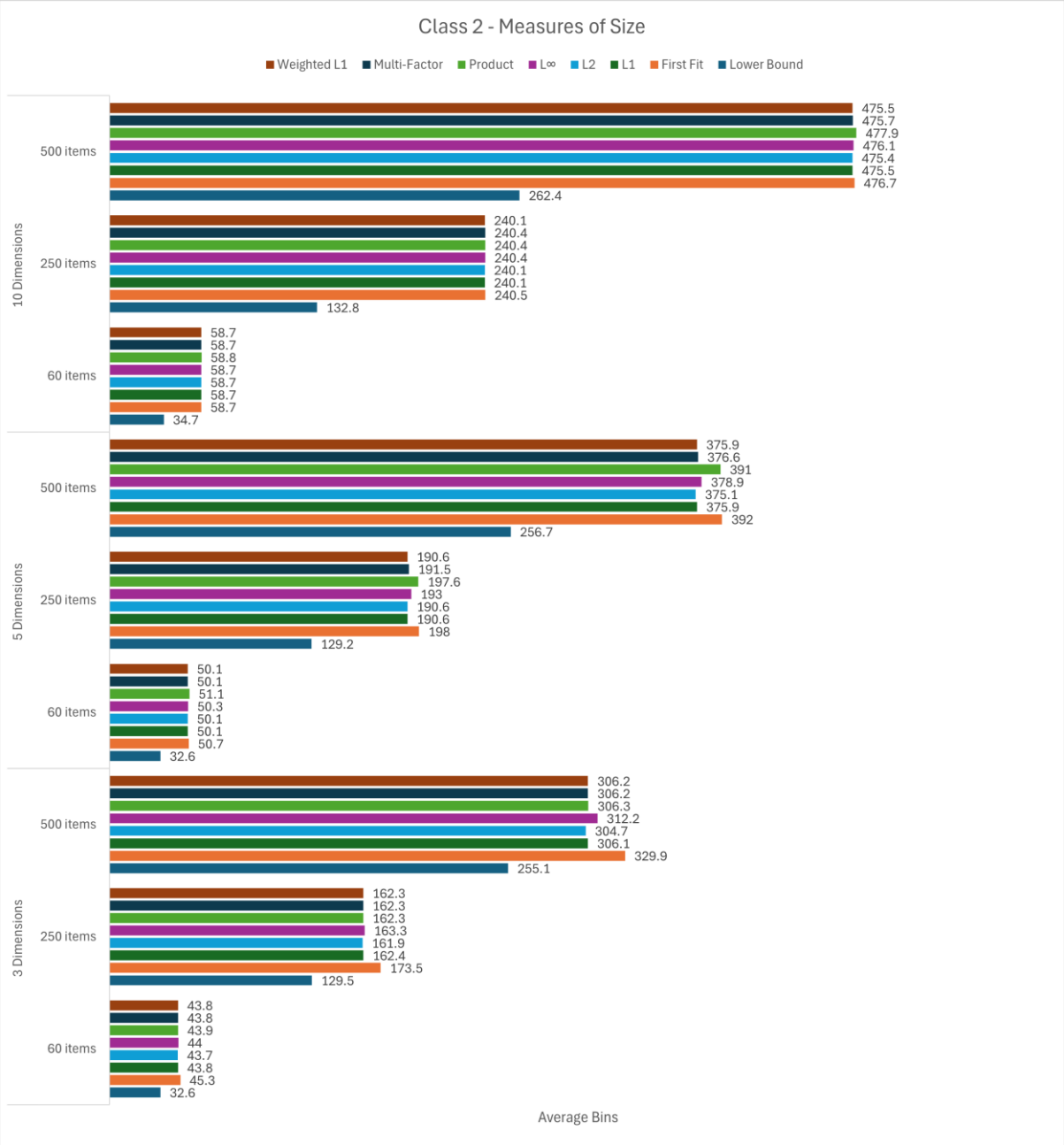


Figure 7 - Comparison of Measures of Size for Panigrahy Class 2

Class 2 reinforces that on a random dataset, the different sort methods perform similarly, however with the exception of the product sort in 5 dimensional cases. This leads to the

conclusion that product sort is not a suitable measure of size, particularly in high dimensional cases.

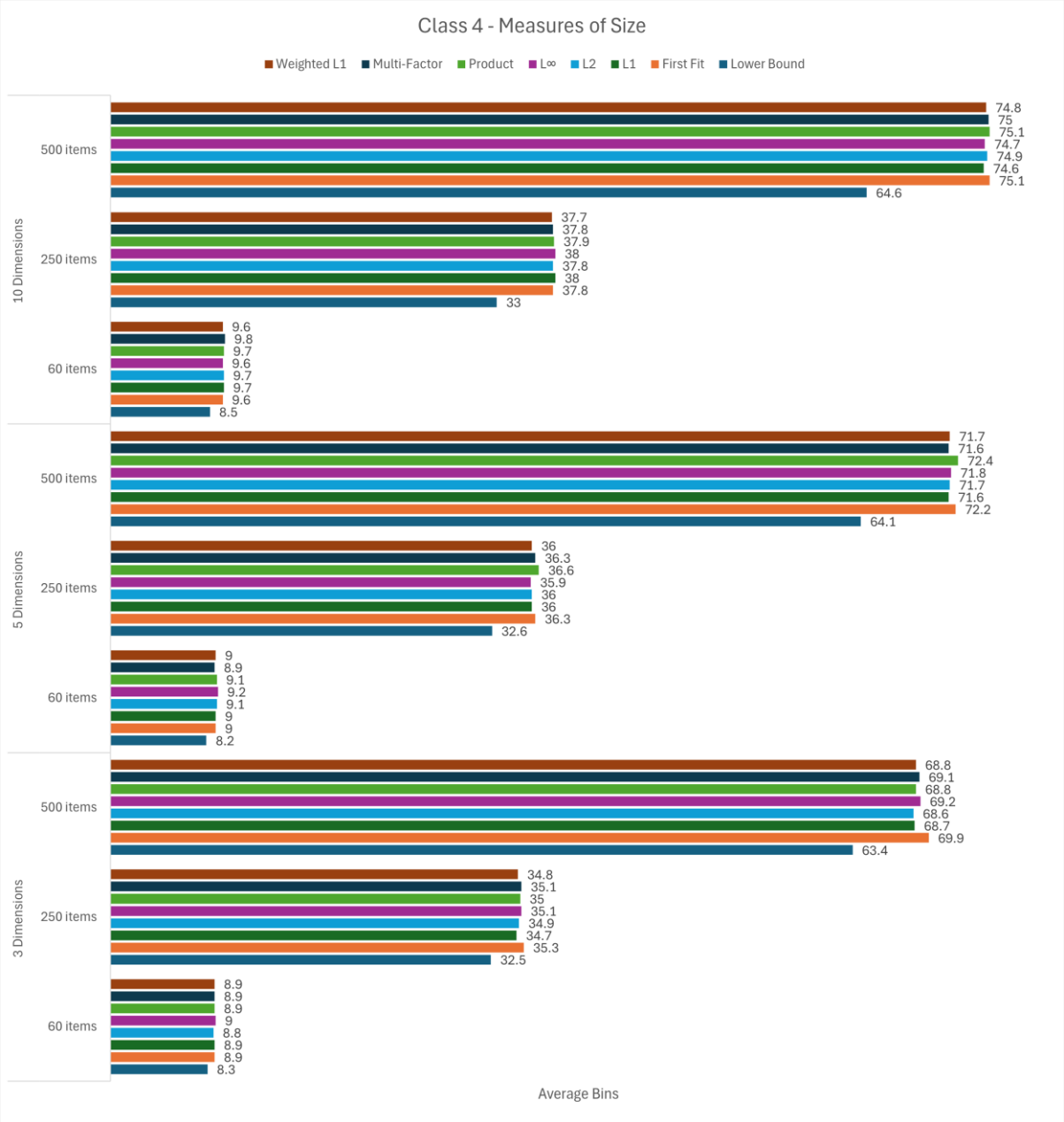


Figure 8 - Comparison of Measures of Size for Panigrahy Class 4

Figure 8 shows that the conclusion from Class 4 of the 2CBP are not applicable to higher dimensional instances. While results remain fairly consistent over the 3 dimensional instances, once 5 and 10 dimensions are used, the gap from first fit to the best sort begins to slightly increase, however it is still far less significant than on instances with larger items. We can see again that product sort is performing poorly, and L1 is particularly good with the very small items.

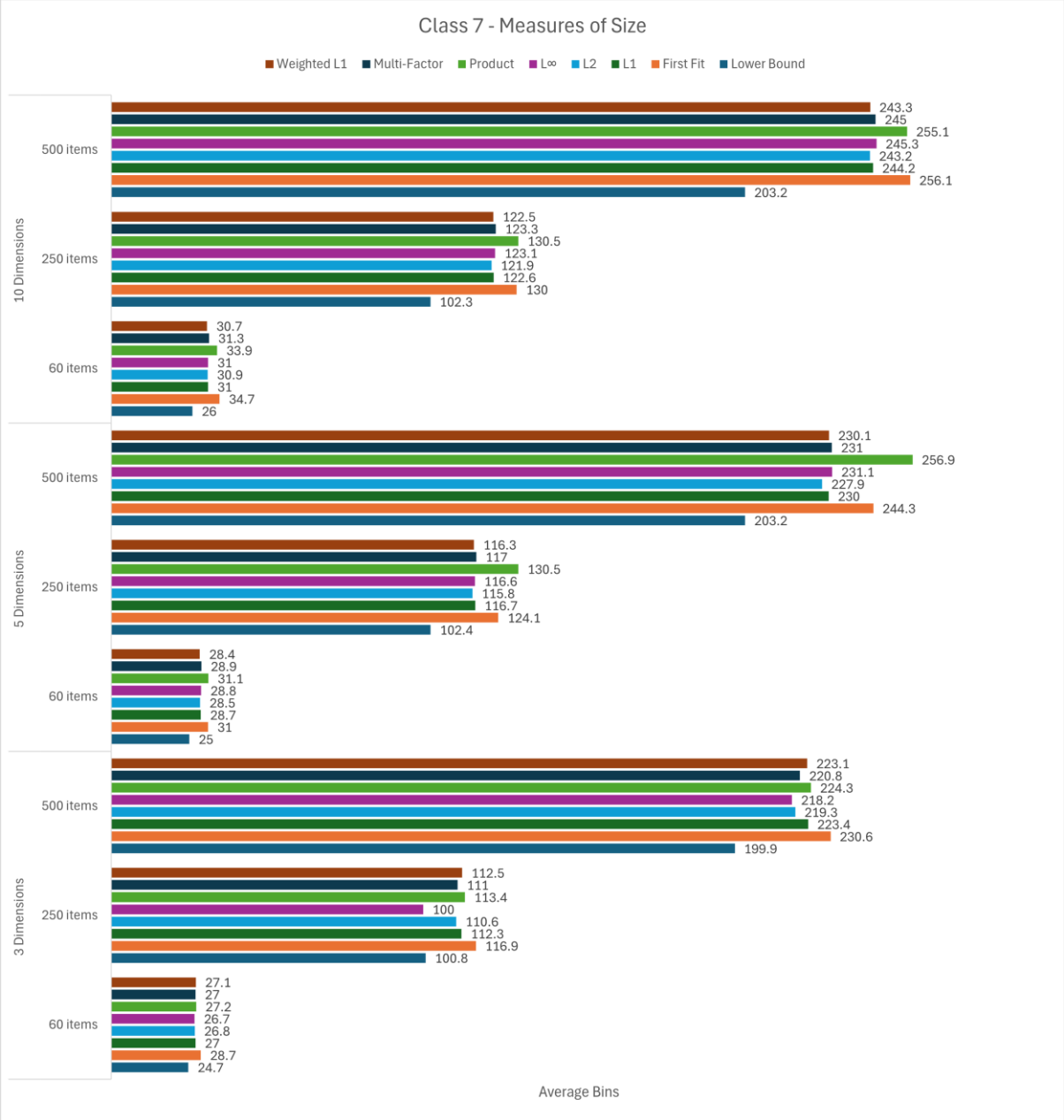


Figure 9 - Comparison of Measures of Size for Panigrahy Class 7

Class 7 has been included to demonstrate just how much worse the product sort can perform than the others, the best example of this is 5 dimensions, with 250 and 500 items in *Figure 9*. In this class, where items are positively correlated, L∞ performs very well. As such, if the dataset is known to include mostly positively correlated items, the L∞ norm may be the most suitable measure of size. Particularly in 3-dimensional instances. L2 also outperforms L1 on Class 7.

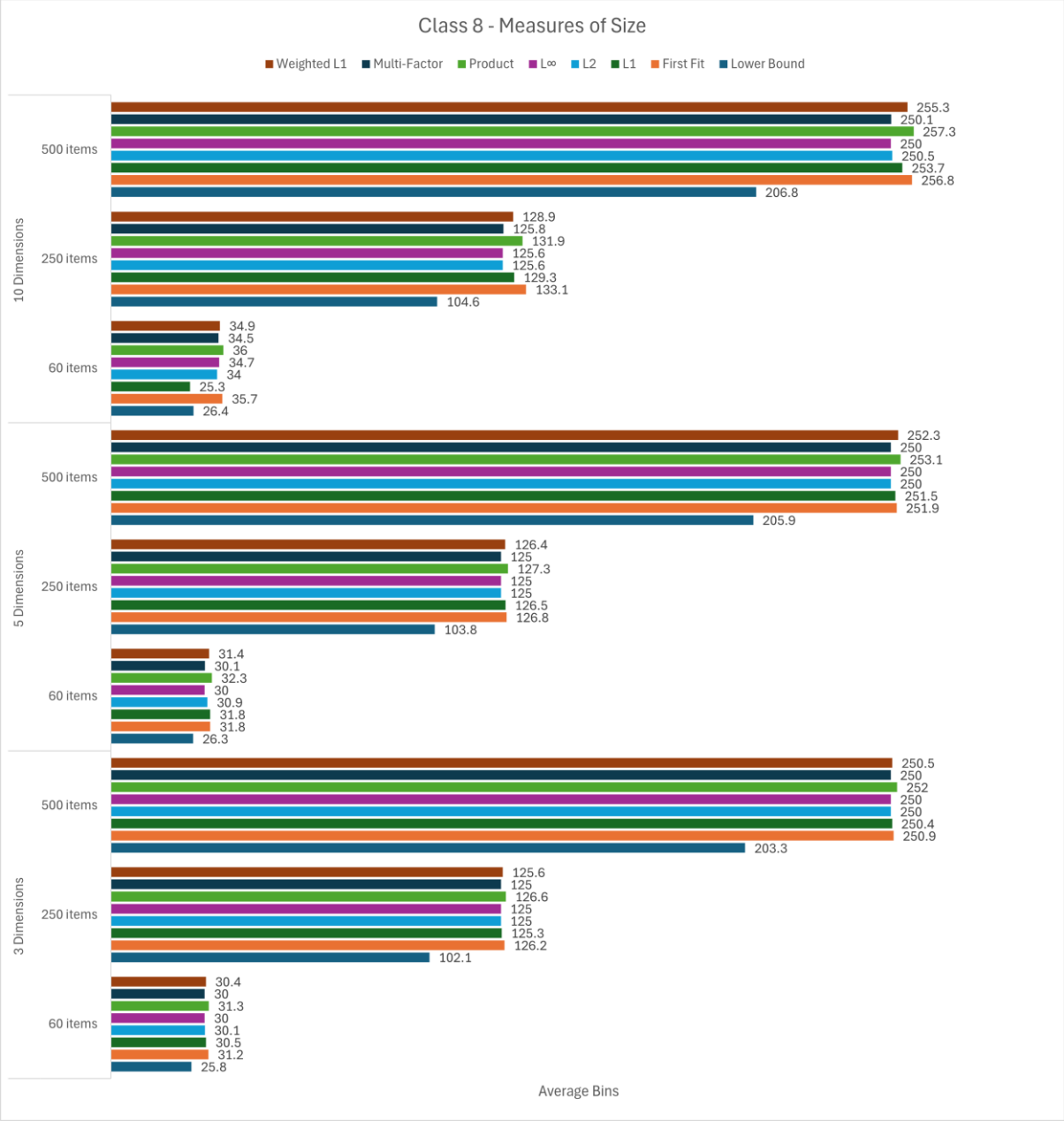


Figure 10 - Comparison of Measures of Size for Panigrahy Class 8

Figure 10 shows Class 8, which gives a similar, but more exaggerated, result to the 2CBP Class 8, with weighted L1 performing poorly, and L2 and L∞ performing well. The multi-factor sort is also effective in these negatively correlated cases.

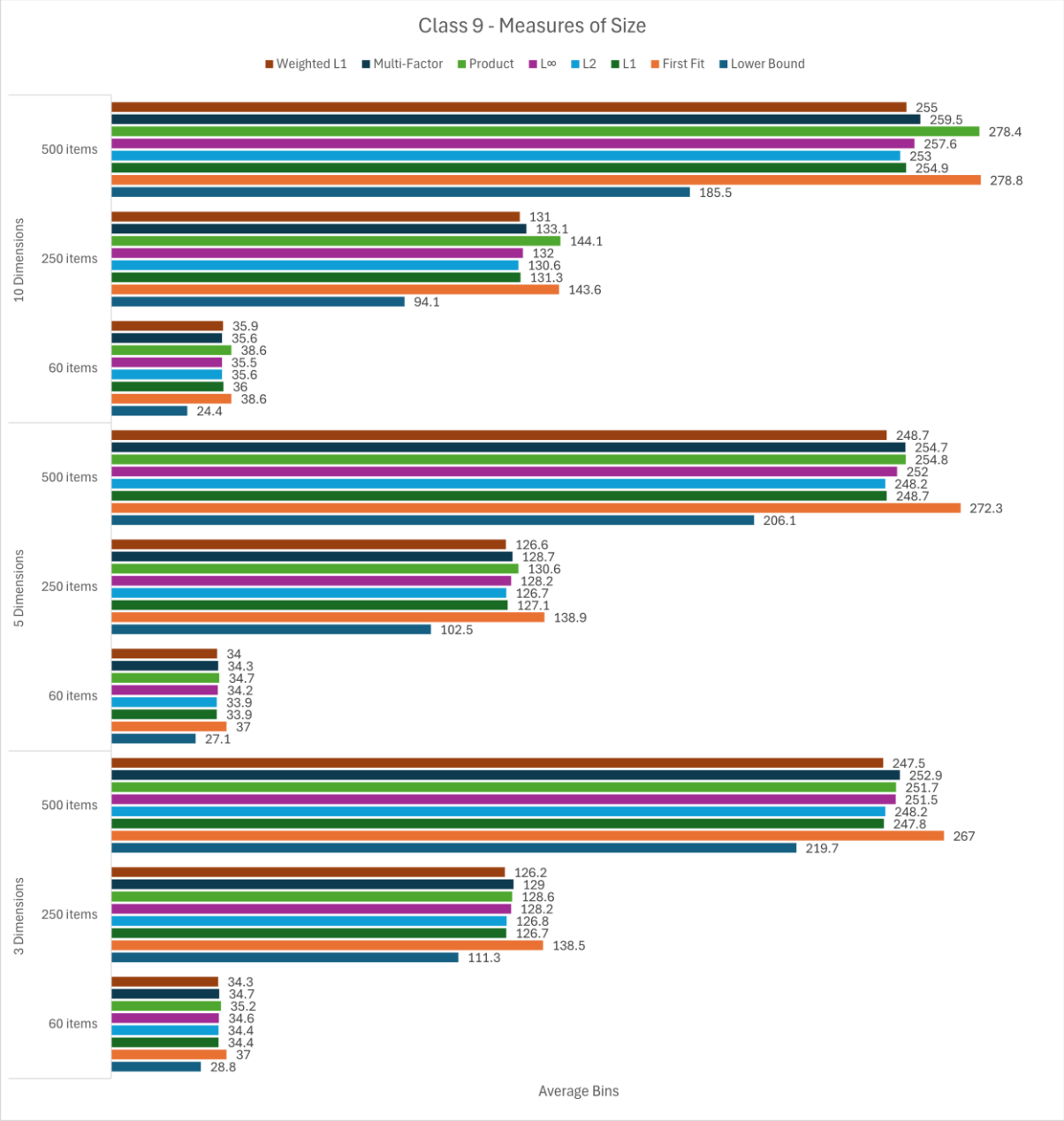


Figure 11 - Comparison of Measures of Size for Panigrahy Class 9

Figure 11 is a prime example of why the unsorted first fit is an unsuitable heuristic for the VBPP, as it is significantly worse than most FFD cases – usually worse by at least 10% of the lower bound. These instances also highlight the similarity in results between L1 and L2.

After the results of the Panigrahy benchmark, the product sort will be excluded from further testing, as it is clearly not a suitable choice.

8.1.3 Mommessin

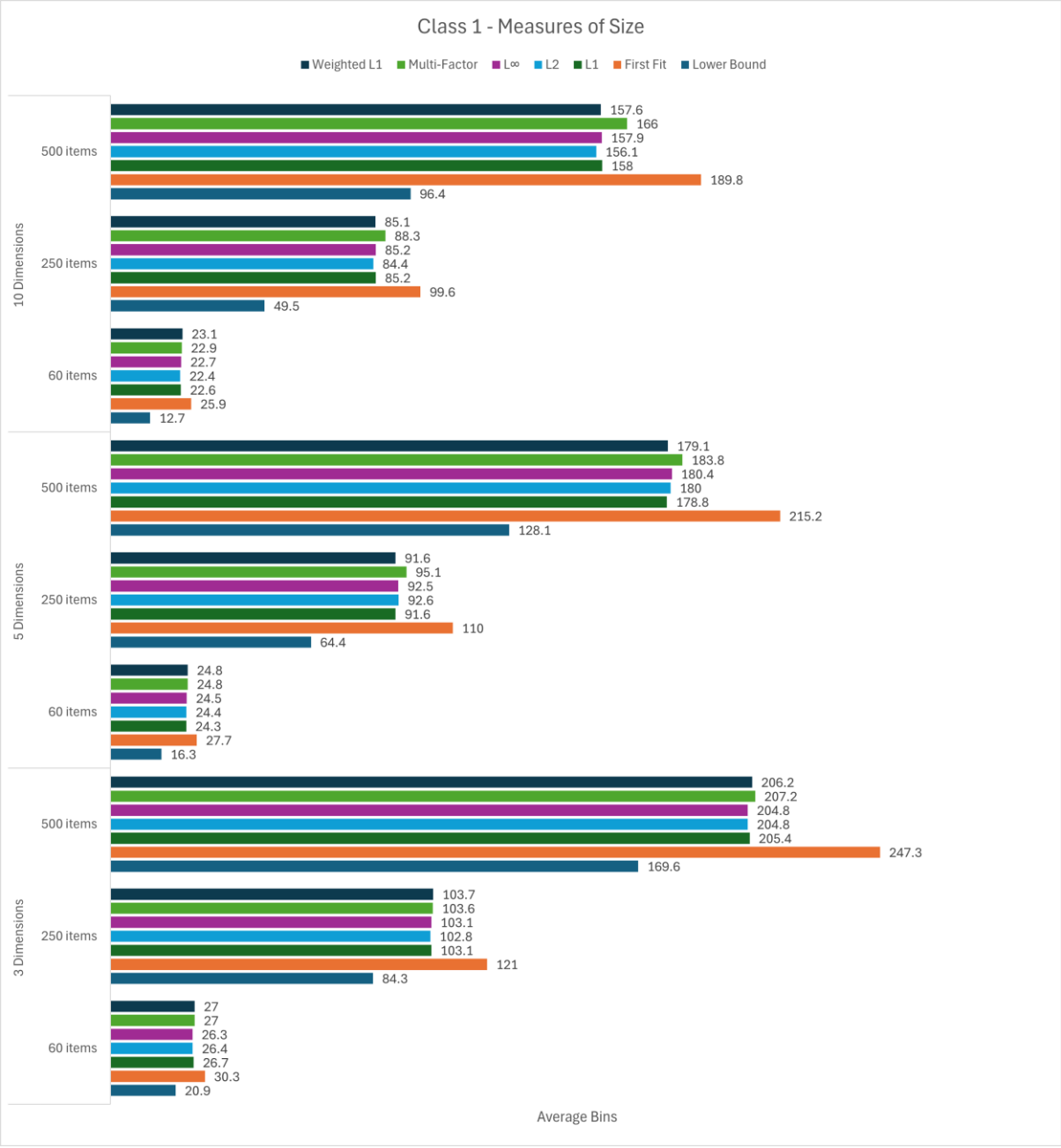


Figure 12 - Comparison of Measures of Size for New Class 1

In *Figure 12*, we can see that the either the L1 or L2 norm provides the best result in most cases. Since the FFD algorithm is fast, it would not be unreasonable to run both if the dataset were not extremely large.

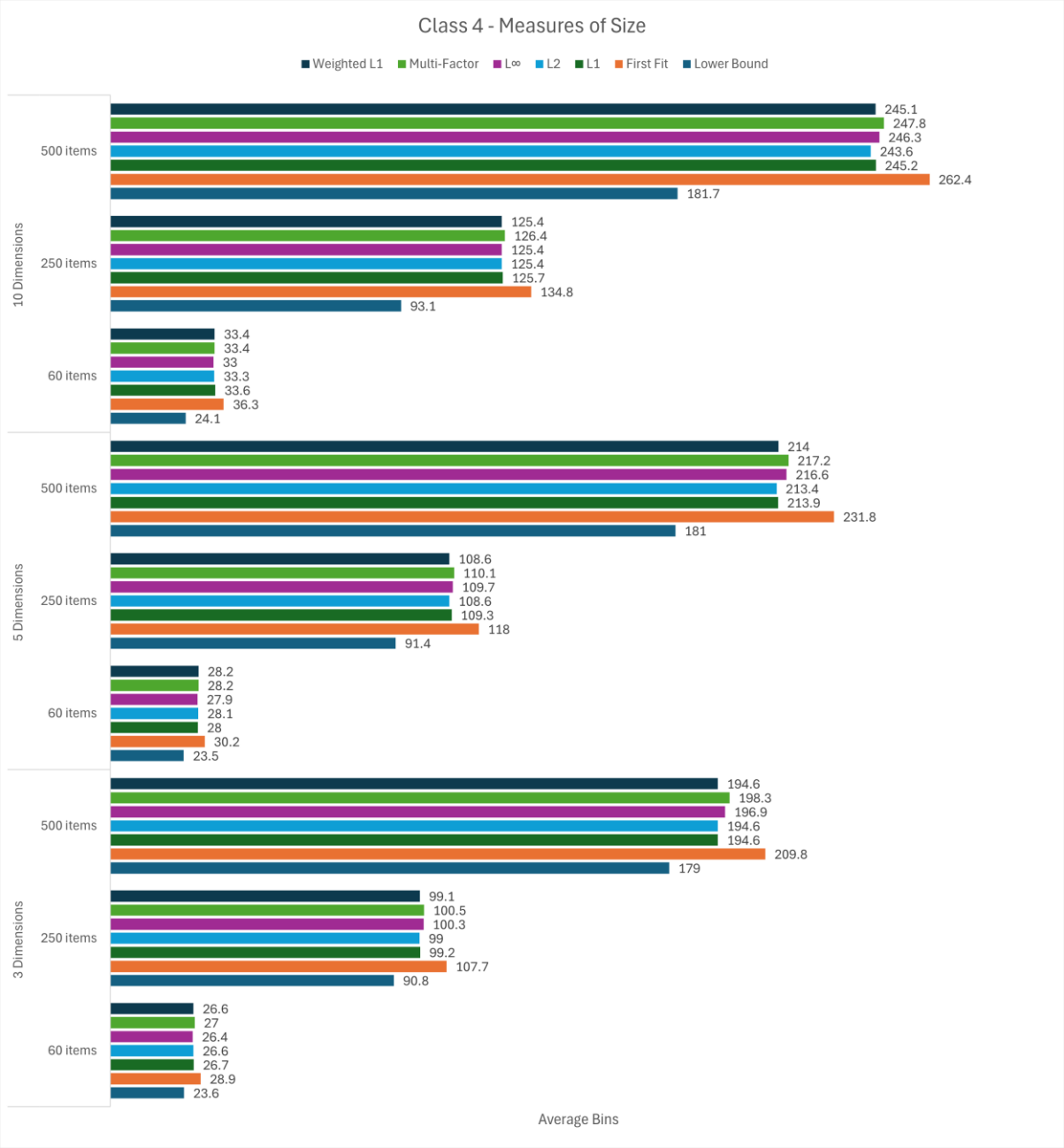


Figure 13 - Comparison of Measures of Size for New Class 4

There is not much of note in *Figure 13*, but it has been included as it gives an example where items are either large in all dimensions, or small in all dimensions. One notable aspect is that the L∞ norm does not perform well on these datasets. This may be explained by the fact that items are similar in all dimensions, but not the same in all. Therefore, one that is slightly larger in one dimension, but smaller in all others, will be given a greater size than one that is consistently large across all dimensions. Since items are balanced, a measure that considers multiple elements is more effective.

8.1.4 Triplet

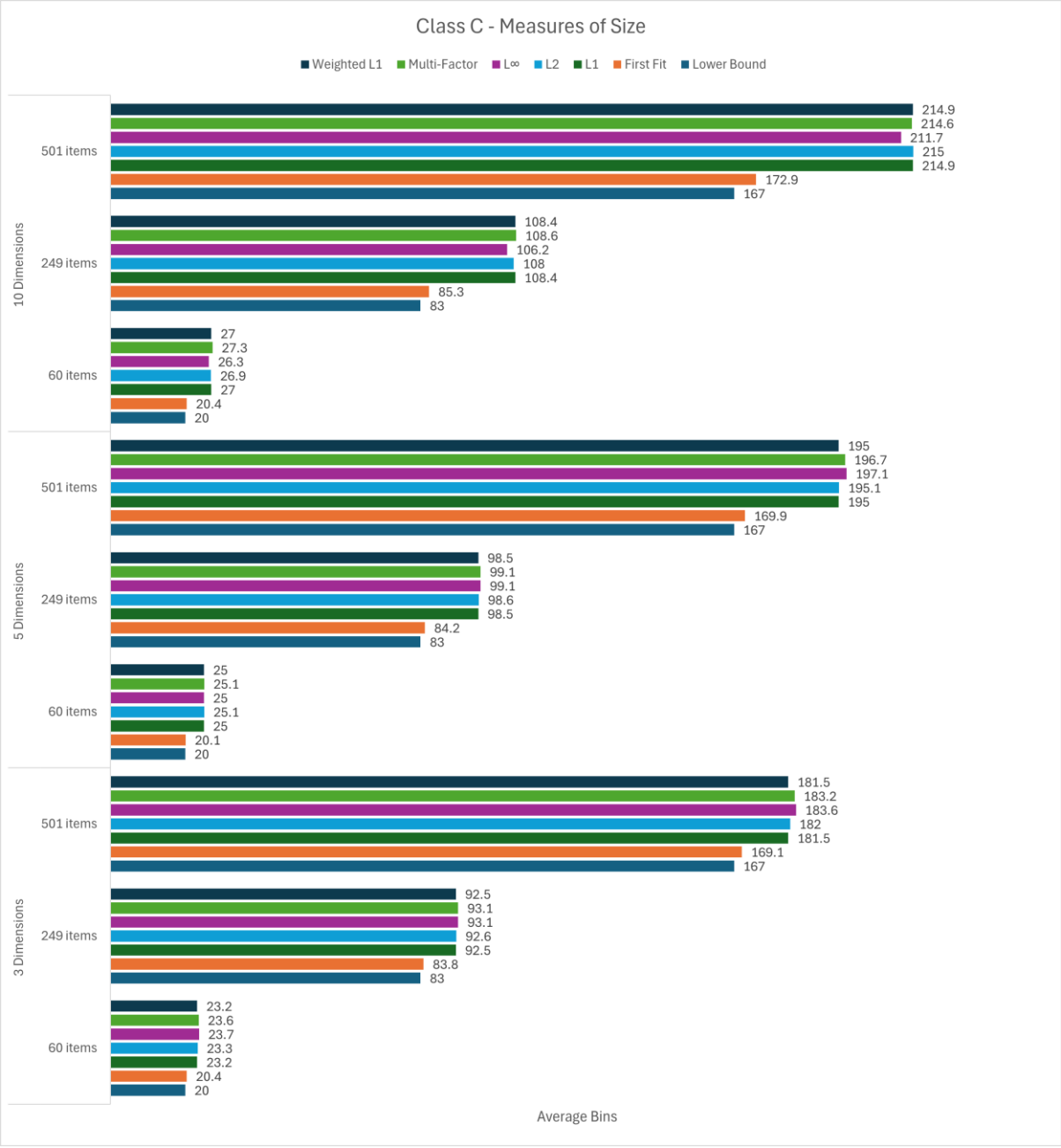


Figure 14 - Comparison of Measures of Size for Triplet Class C

The results in *Figure 14* have one unique feature, in that the first-fit massively outperforms all FFD instances. This highlights an important point about context when implementing the VBPP, or any approximation algorithm – it is important to consider the nature in which the dataset it generated, as well as the nature of the dataset itself. Because this benchmark has been generated in triplets that fit perfectly into bins, the items that fit best with each other, are already next to each other in the dataset. Because of this, not changing the order means

that far more bins can be optimally filled. While this result clearly doesn't prove that first-fit is a superior heuristic, it does show that if data is from a source that will result in certain patterns, it is worth considering a more unique approach, rather than one of the general heuristics designed to work on most datasets. This class does have a strange characteristic in that L∞ gives particularly poor results in 3 and 5 dimensional instances, yet in 10 dimensions, it gives the best results of any of the sorting methods.

Overall, one of the L1, L2, and L∞ norm usually gives the best, or close to the best result. Since these are very simple, and not computationally demanding, it could be recommended to test all 3 of these and choose the best packing in an implementation of FFD. L1 gives the most consistent results, L2 and L∞ are still very consistent, but fluctuate between better/worse than the L1 norm. If the nature of the dataset is unknown, the L1 norm is a strong choice as it is extremely stable in its results, and if the nature of the dataset is known, the measure of size can be chosen based on the evaluation of *Figure 1-14.*

8.2 Item-Centric vs Bin-Centric

This section will provide a brief overview of the time performance of the item-centric and bin-centric FFD heuristics with the L1 norm as the measure of size. It is expected that item-centric will by far outperform bin-centric in terms of time. The first-fit algorithm will be used as a control.

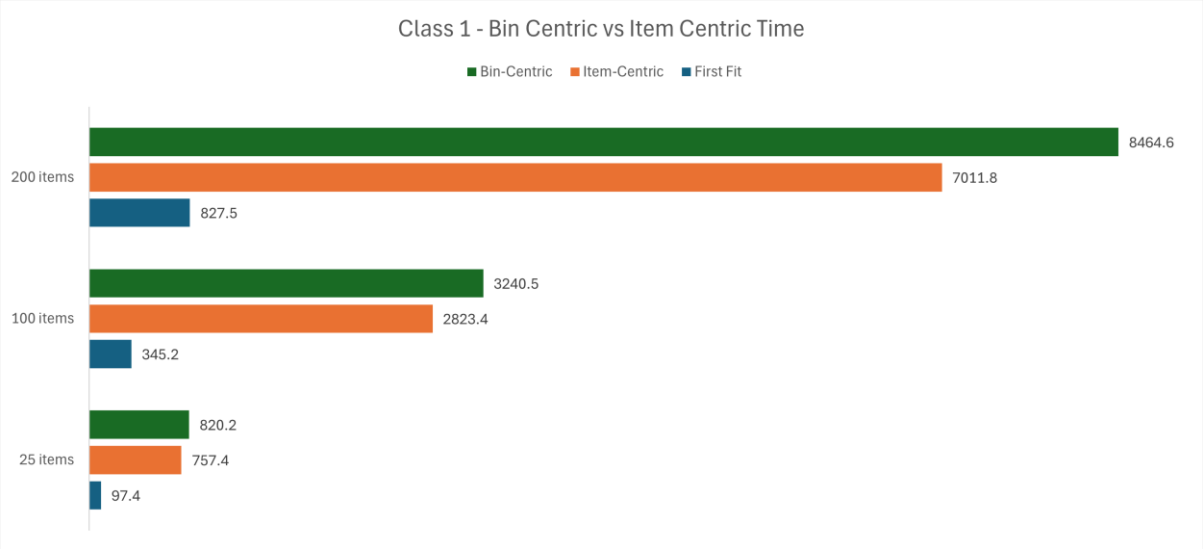


Figure 15 - Bin Centric vs Item Centric Time Performance 2CBP Class 1

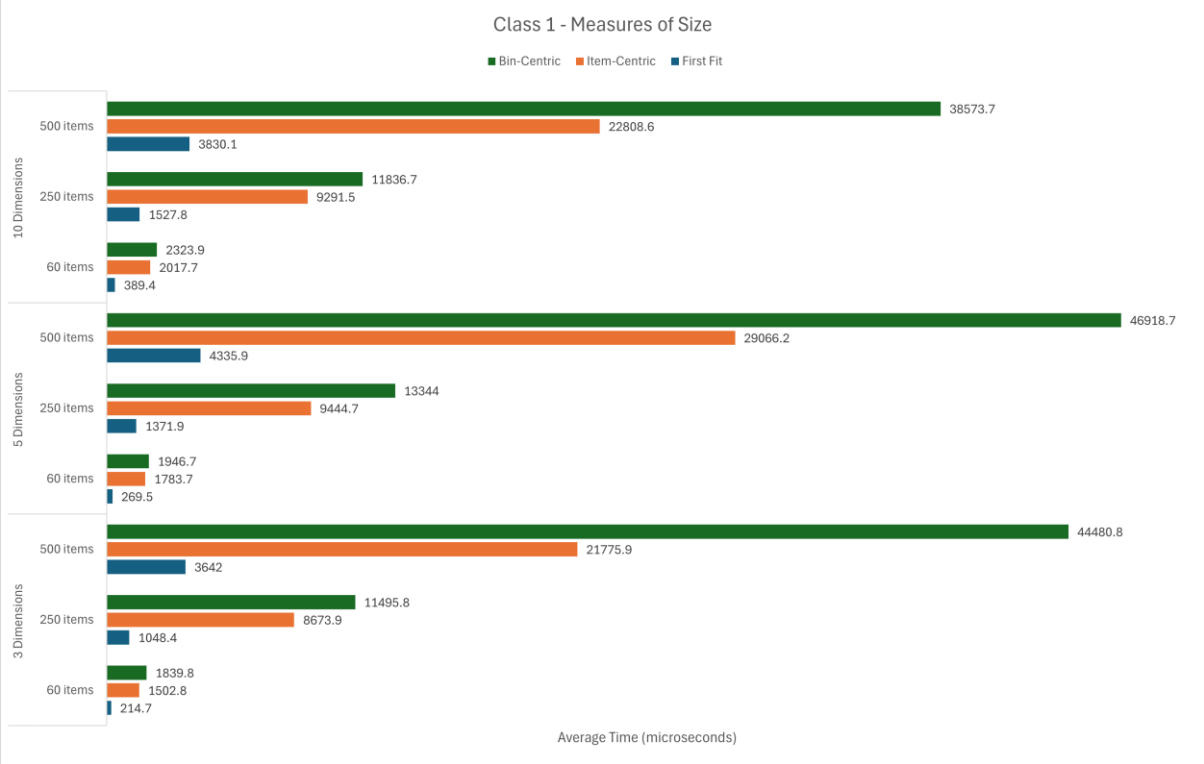


Figure 16 - Bin Centric vs Item Centric Time Performance Panigrahy Class 1

As seen in *Figure 15, 16*, the item-centric FFD is much faster than bin-centric FFD. This result was expected, but one trend that was less expected is that from 3-dimensions, as the number of dimensions increases, the gap in time decreases. In 3 dimensions with 500 items, bin-centric takes over double the time that item-centric takes, but there is only a $\sim 60\%$ increase with 10 dimensions and the same number of items. A plausible reason for this is that the sorting algorithm is taking a larger proportion of the time in these higher dimensional instances, meaning that as more dimensions are added, the times should begin to converge. This experiment acts as evidence that the item-centric approach is less computationally demanding and should therefore be chosen over bin-centric if possible. This is however not possible with heuristics such as the dot product heuristic introduced by (Panigrahy, et al., 2011).

8.3 Comparison of Heuristics

The main focus of this research has been into existing and new heuristics. Below is an analysis of the results of each heuristic. The same benchmark classes that were used to test measures of size will be used. In the graphs, the bars represent average bins used, and lines represent time taken.

8.3.1 2CBP

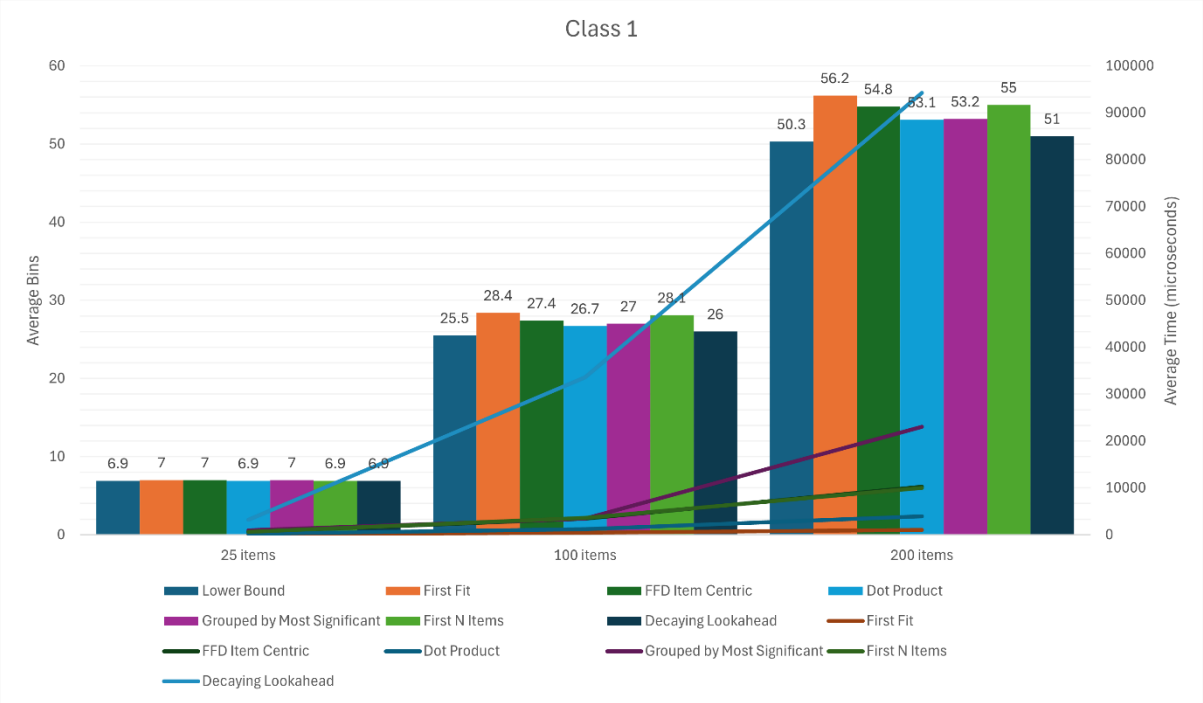


Figure 17 - 2CBP Class 1

From *Figure 17,* we can see that the newly introduced Decaying Lookahead heuristic produces the best results, with the average packing always within 0.7 bins of the lower bound. For the small dataset, almost every packing was optimal, but in the larger instances, the best performing heuristics were Dot Product and Decaying Lookahead. With the exception of first fit, which consistently produced the worst results, Dot Product was the fastest heuristic on all 3 sizes, and Decaying Lookahead was the slowest, however this reached a maximum of $< 100000\mu s = 0.1s$ on a set of 10 instances, giving an average $\sim 0.01s$ on datasets of size 200, so this can still be considered a reasonable time. Based on the observed trends, it may not scale up to datasets in the 10000s of items or more.

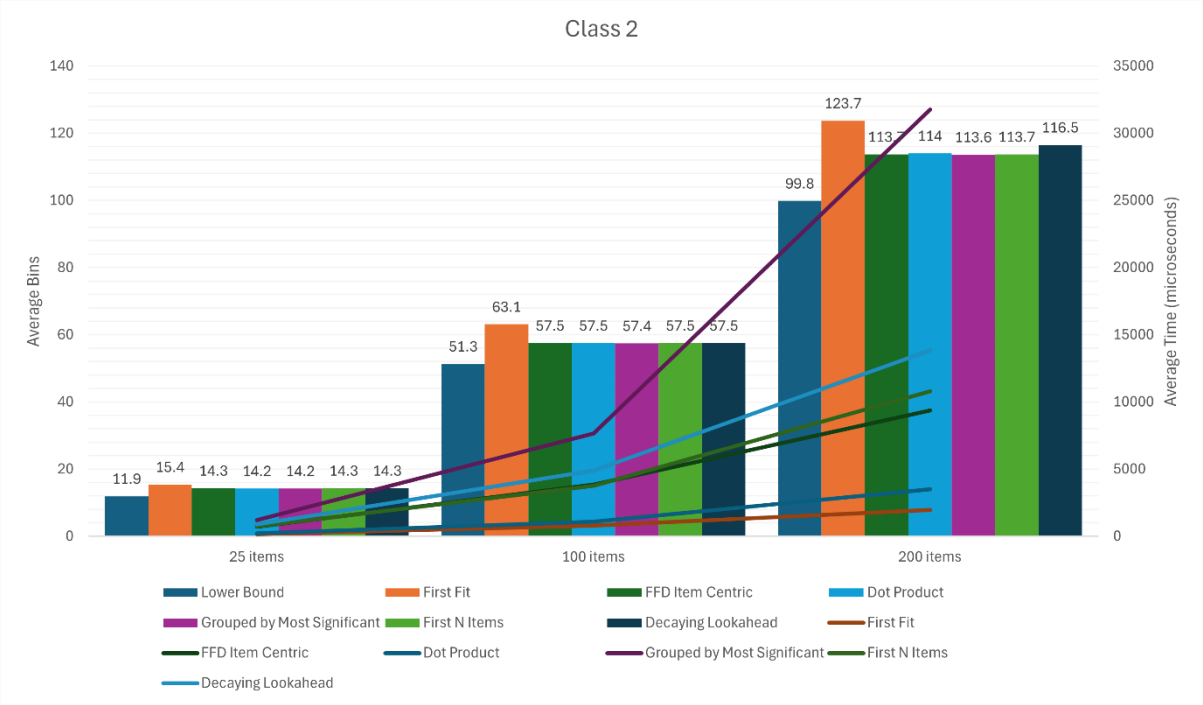


Figure 18 - 2CBP Class 2

Class 2 gave very similar results for the heuristics, with Grouped by Most Significant having marginally better results than the others (better than next best by no more than 0.1 bins), but at the sacrifice of longer runtime time. Therefore, if the dataset is of unpredictable nature, the Grouped by Most Significant or Dot Product heuristics would make suitable choices, if you are prioritising tight packings or fast runtime, respectively.

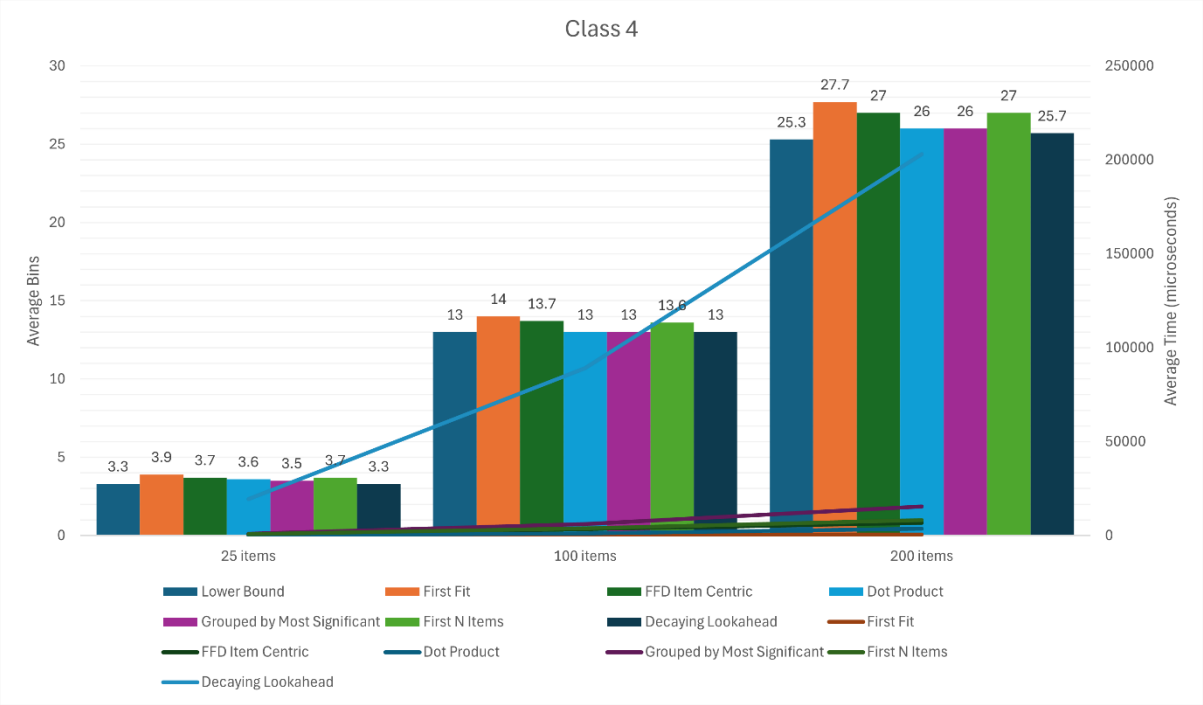


Figure 19 - 2CBP Class 4

Figure 19 shows class 4 of the 2CBP dataset, which contains very small items. In this instance, we can see that the Decaying Lookahead heuristic finds perfect solutions for ≤ 100 items, and very close to optimal for the largest dataset. However this comes at a huge performance cost, and the Dot Product and Grouped by Most Significant find almost as good of a result in a fraction of the time. If the priority is a tight packing, with no time concerns, Decaying Lookahead can provide exceptional results if the dataset consists of very small 2-dimensional items.

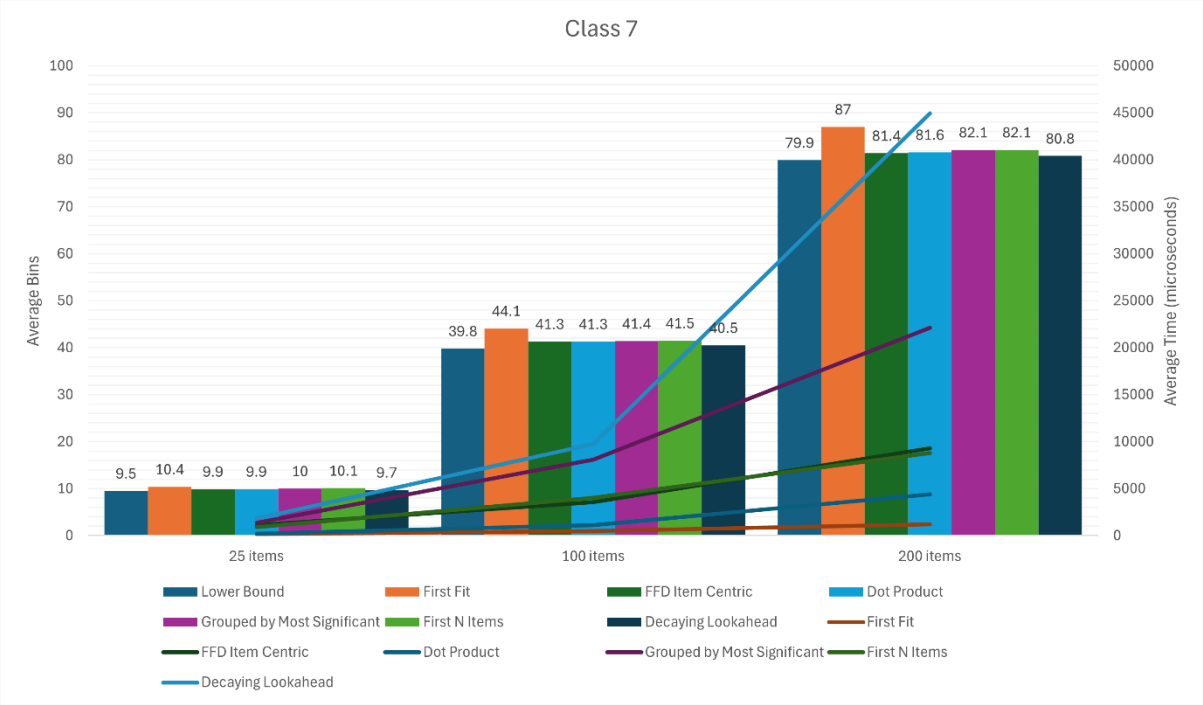


Figure 20 - 2CBP Class 7

The class used in *Figure 20* contains positively correlated items. The Dot Product performs with a good balance of time and performance, with the Decaying Lookahead giving the best results in a slightly longer time ($\sim 2000\mu s$ per dataset on the largest instances). The time for the Decaying Lookahead only becomes noticeably greater than the others on the large dataset, and it gives a result 1.1 bins greater than the lower bound. As expected, the Grouped by Most Significant does not do well on this type of dataset, as items are positively correlated, so the decision of which item to place is more to do with the size of the whole vector, rather than its most significant element.

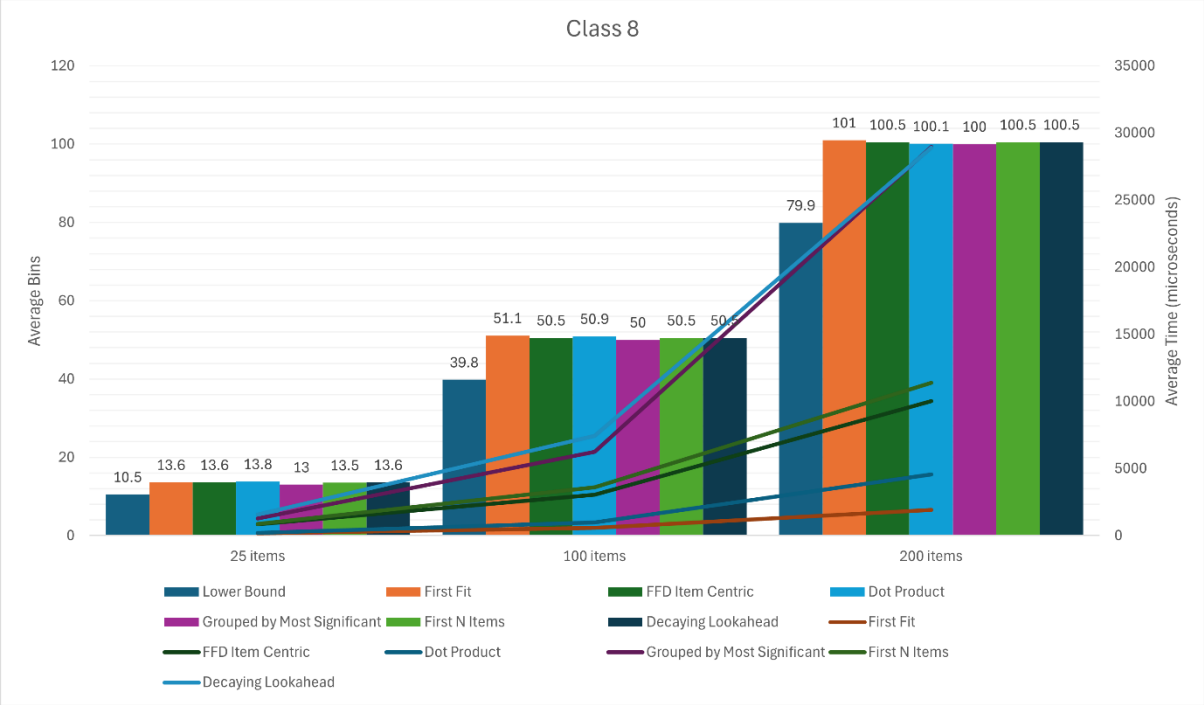


Figure 21 - 2CBP Class 8

Class 8, as predicted in 5.2.4, is suited well to the Grouped by Most Significant heuristic. It takes marginally less time than the Decaying Lookahead and gives the best result for all dataset sizes. The other heuristics all give a result within 0.6 bins of each other.

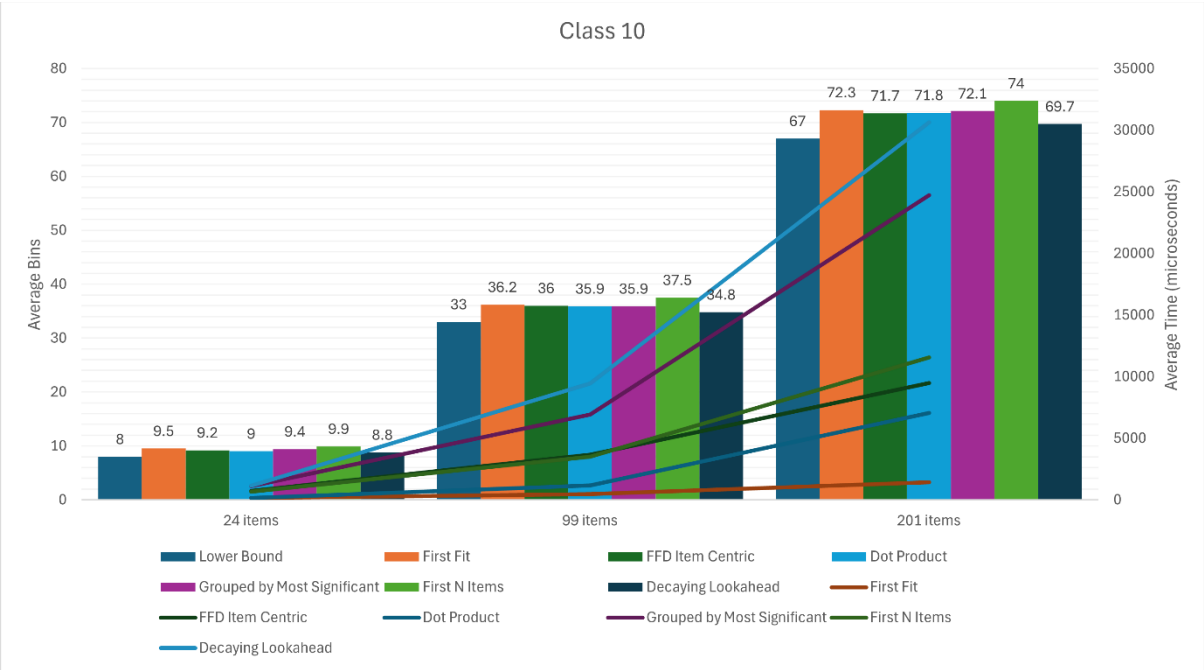


Figure 22 - 2CBP Class 10

Figure 22 shows Class 10, which has triplets of items that fit perfectly into bins. The Decaying Lookahead heuristic performs very well here, especially on the larger datasets, with an average of 2 less bins than the next best heuristic, and only 2.7 bins away from the lower bound. This comes at a computational cost, but not as significant as on other datasets.

8.3.2 Panigrahy

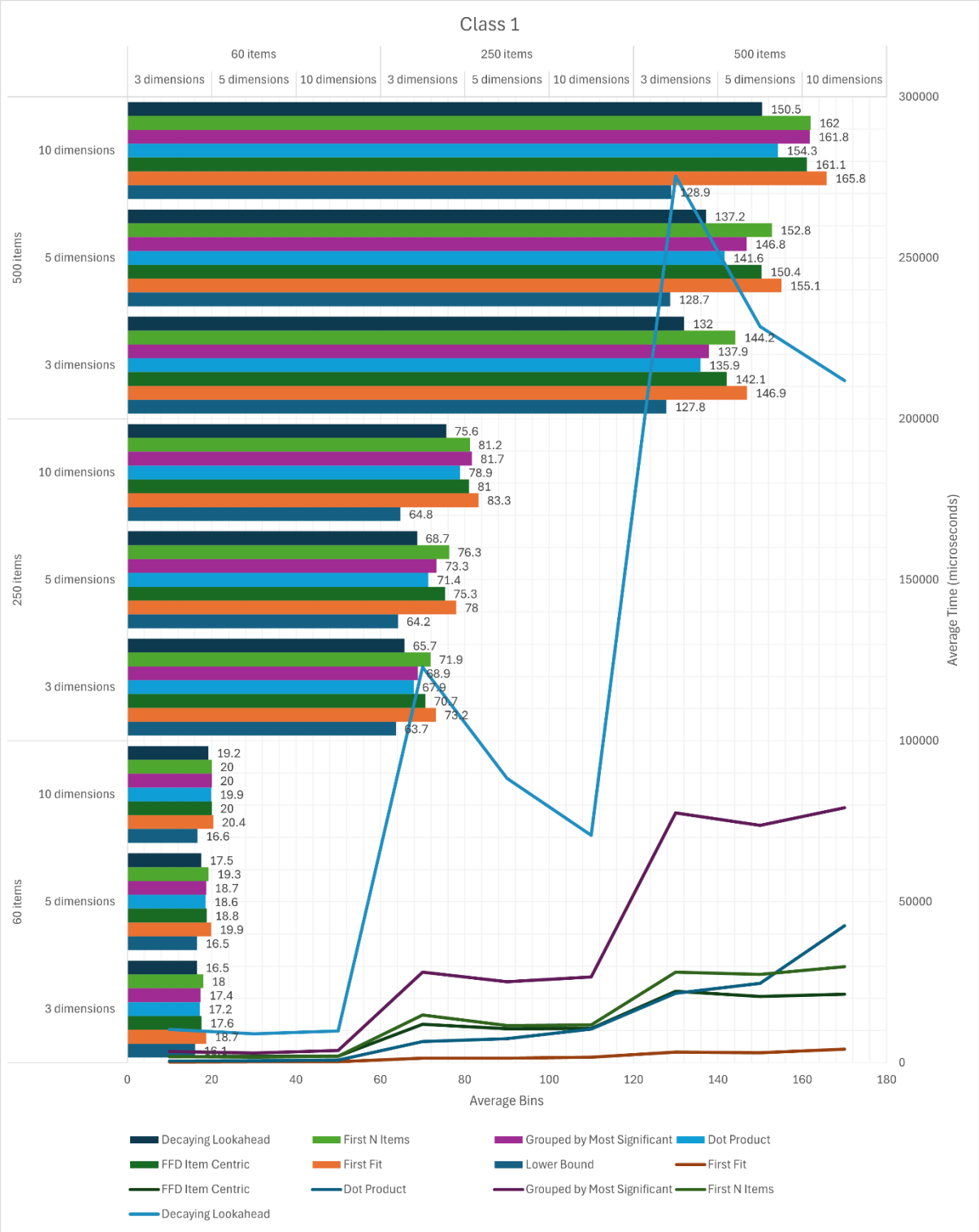


Figure 23 - Panigrahy Class 1

The results from Class 1 of the Panigrahy benchmarks are similar to Class 1 of the 2CBP, but the difference in results is greater. The Decaying Lookahead heuristic produces much better results, as close as 5.8 bins away from the lower bound, at a huge computational cost on

large datasets, and a better result in slightly more time on the smaller datasets. The number of dimensions appears to have a negative correlation with time for the Decaying Lookahead. The next best heuristic in terms of results is the Dot Product, which runs with one of the best times, until the large datasets with many dimensions, where there is a spike in time.

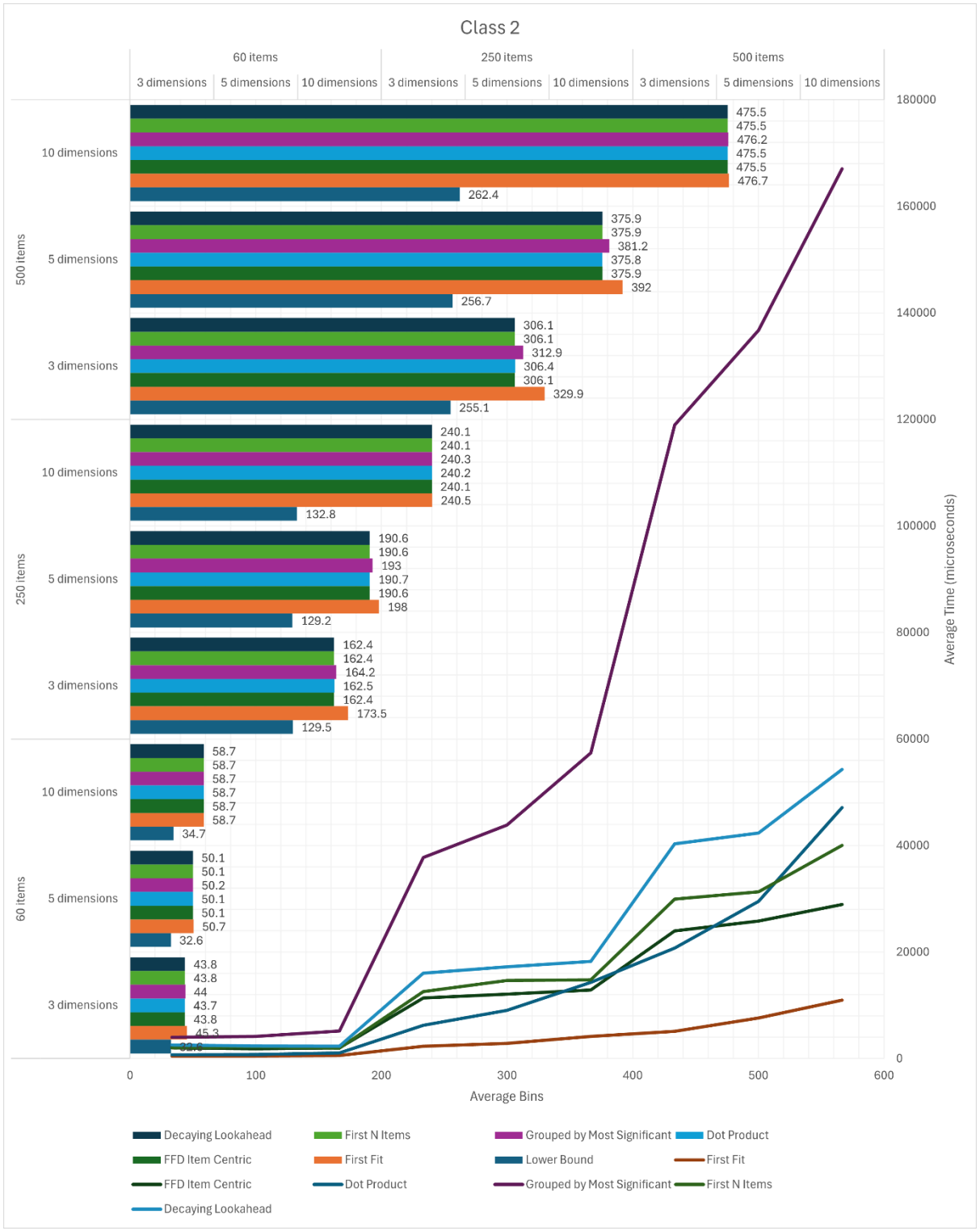


Figure 24 - Panigrahy Class 2

In *Figure 24*, we can see that the Decaying Lookahead does not run in much more time than the other heuristics (at worst $\sim 3000 \mu s$ worse than next slowest per dataset), which is a change from what has previously been seen. The Grouped by Most Significant heuristic takes much more time on this dataset, and produces a worse result, meaning it is poorly suited to this type of dataset. The others all perform very similarly, with Dot Product and First N Items being similarly stable in results and time.

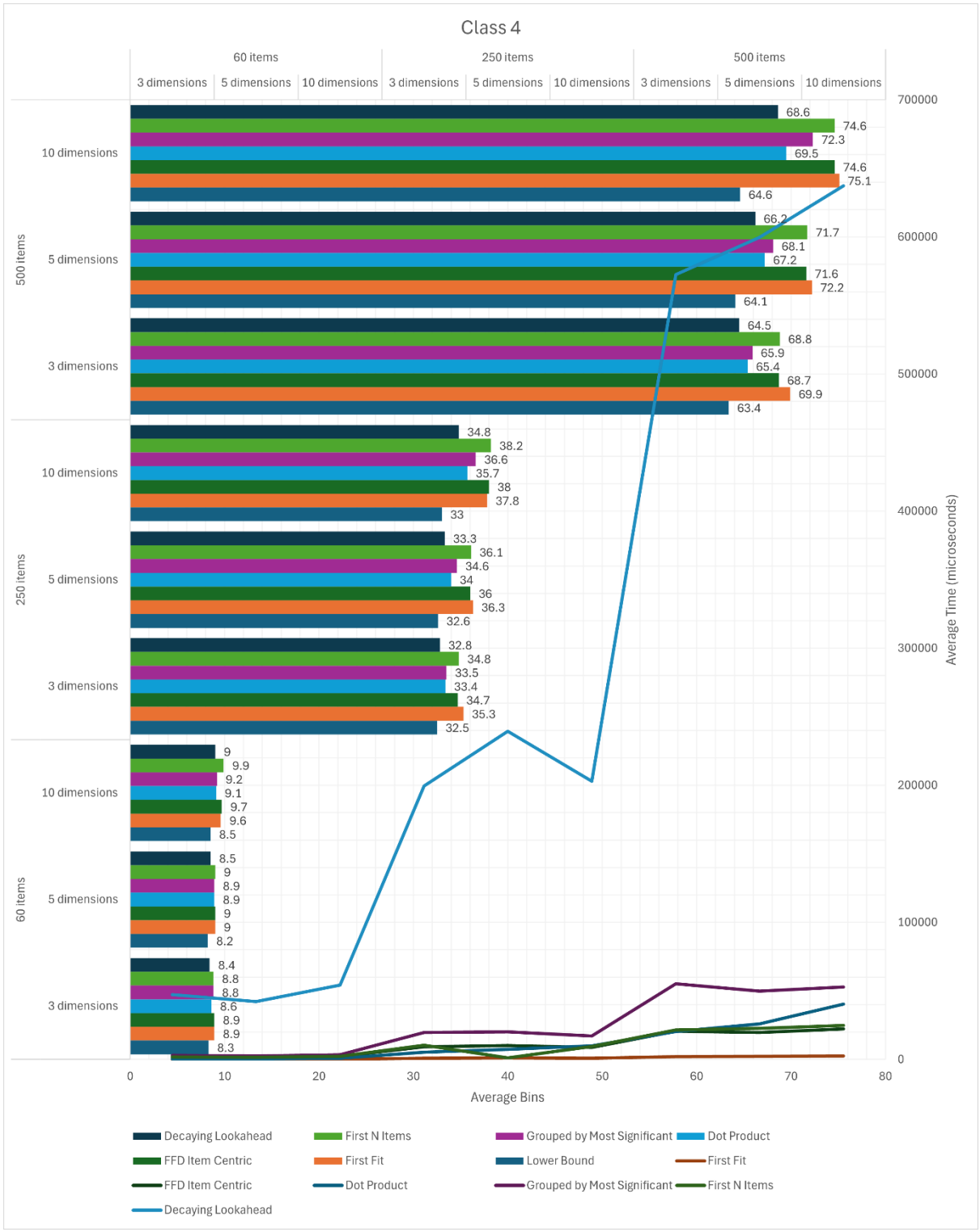


Figure 25 - Panigrahy Class 4

Figure 25 reinforces the ideas from *Figure 19* that the Decaying Lookahead heuristic is very good at finding a tight packing for a set of very small items. Since this benchmark has more dimensions and larger datasets, we can see that the time performance gets increasingly worse as the number of items increases but does not appear to have a correlation with the

number of dimensions. While the maximum time taken is a factor of 10 greater than the other heuristics, it is still only $\sim 65000 \mu s$ per dataset, which is still under 1 second.

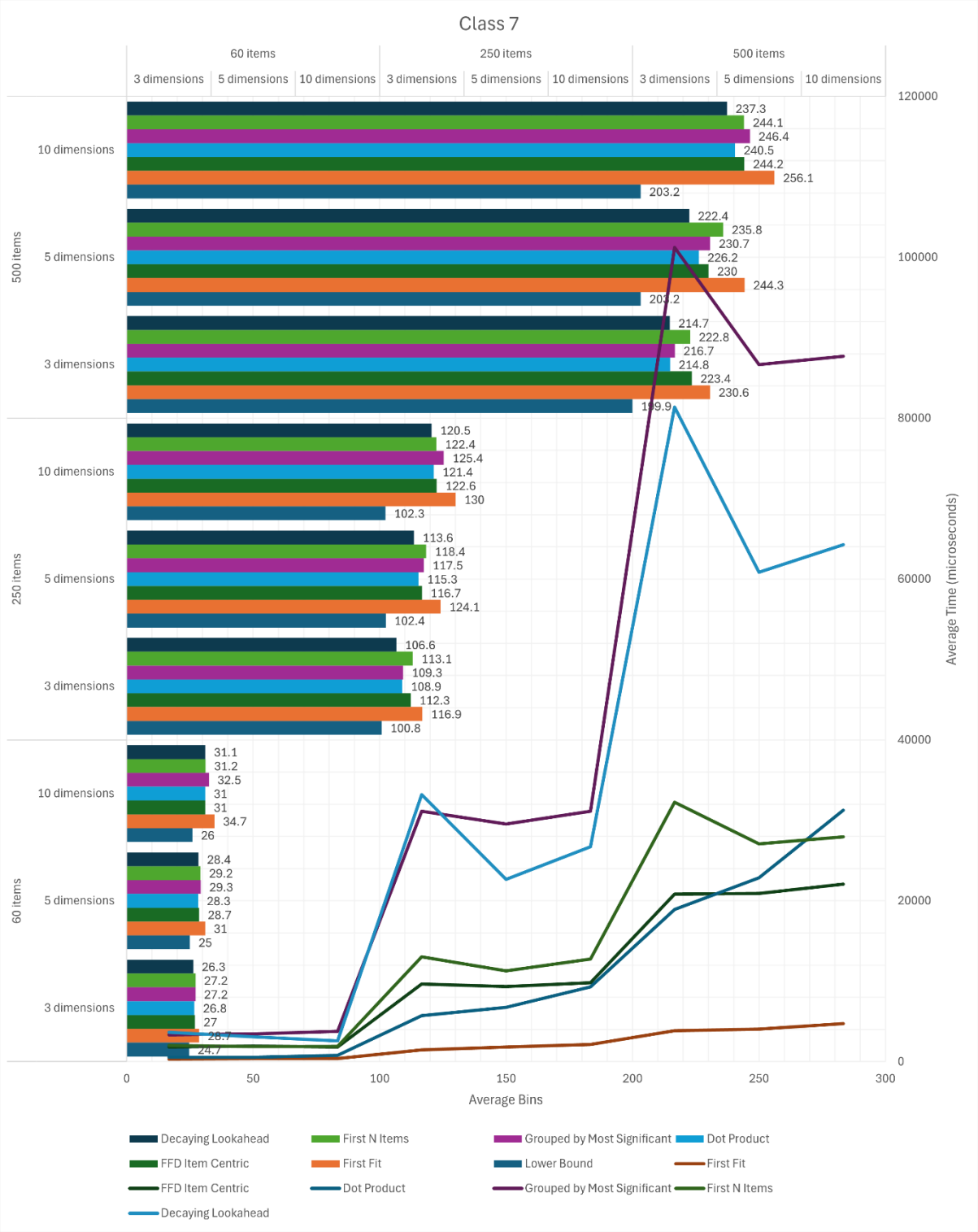


Figure 26 - Panigrahy Class 7

Similarly to Class 7 of the 2CBP benchmark, Grouped by Most Significant performs poorly and has a long runtime. Dot Product performs well in the smaller datasets, and Decaying

Lookahead gives the best results in the larger datasets. First N Items also gives a poor result on this dataset, but gives a quick result, which was one of the design ideas behind the heuristic. FFD does outperform it on both time and result though, making First N Items an unsuitable choice here.

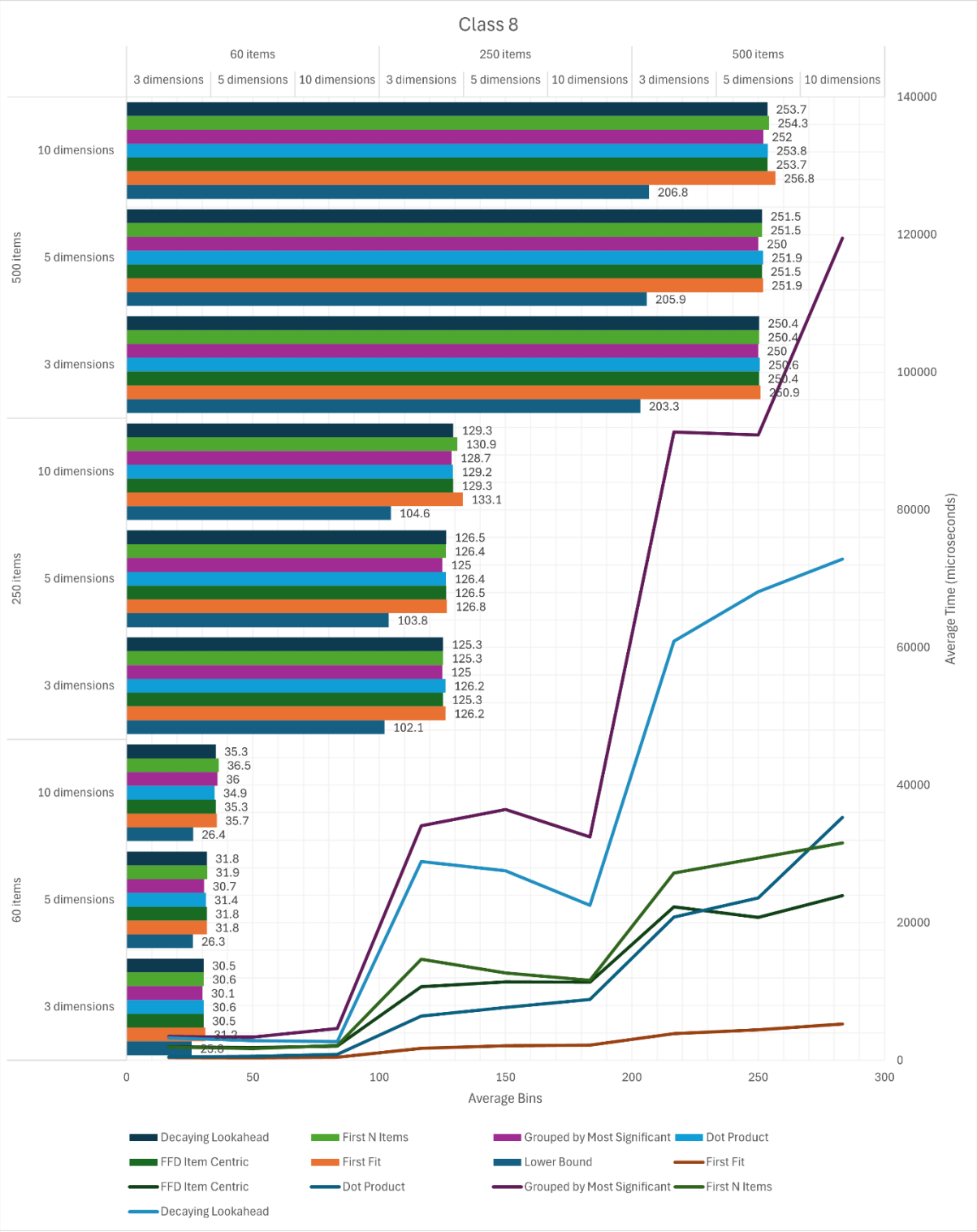


Figure 27 - Panigrahy Class 8

As with Class 8 of 2CBP benchmark, Grouped by Most Significant works well on this dataset, but takes a long time to get to the result. It gives a result up to 1.8 bins better than next best. The Dot Product gives a marginally worse result than Decaying Lookahead, but in approximately one quarter of the time, making it a suitable choice for when a balance between time and performance is required.

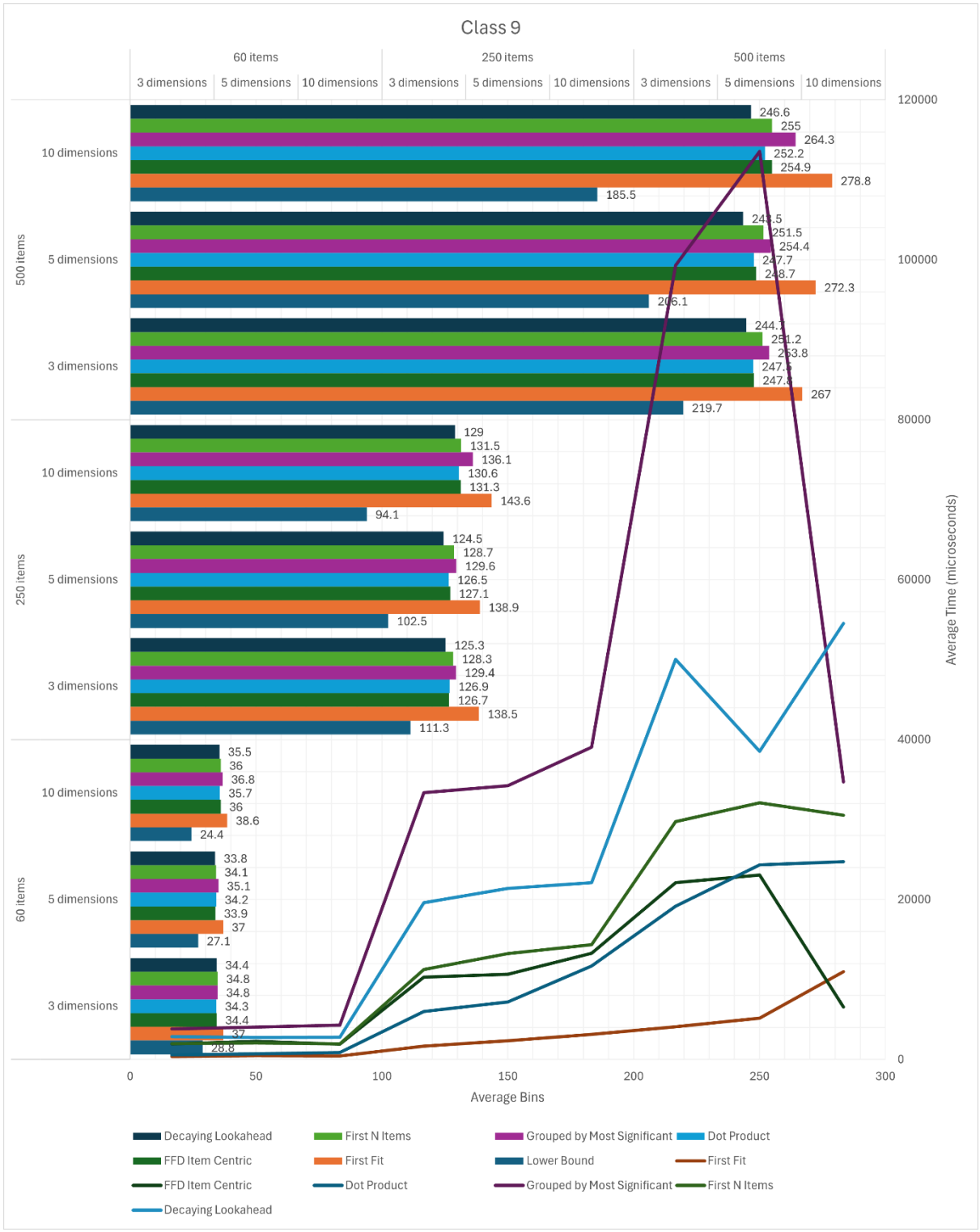


Figure 28 - Panigrahy Class 9

Contrary to the prediction, Grouped by Most Significant performs poorly on this dataset. The best for this are Dot Product and Decaying Lookahead, with Decaying Lookahead producing most of the best results, and not as slowly as it is on some other benchmarks where it gives the best results.

8.3.3 Mommessin

The classes 1 and 4 introduced by (Mommessin, et al., 2025) contain items large in one dimension and balanced items, respectively.

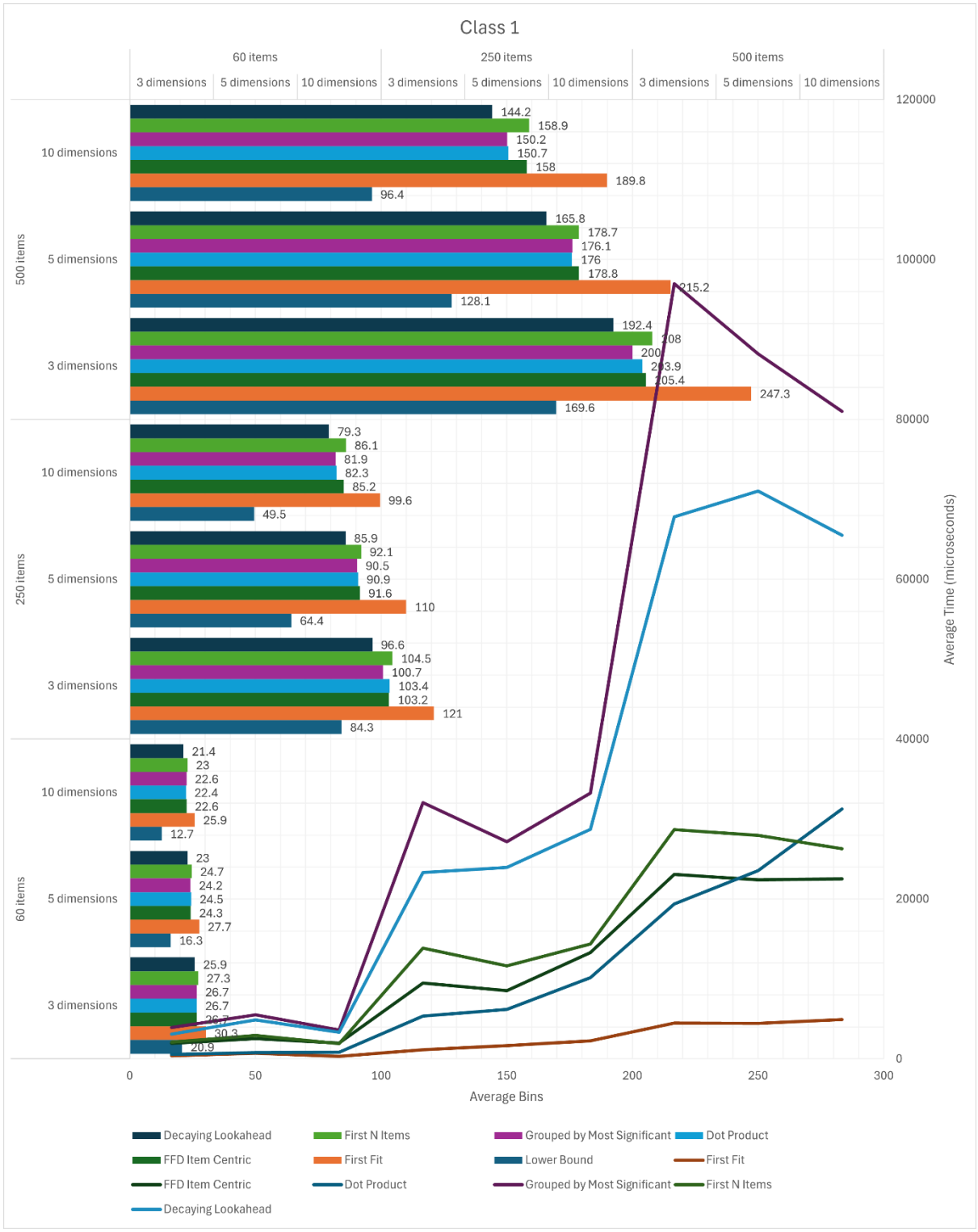


Figure 29 - New Class 1

The results from *Figure 29* show that the Decaying Lookahead gives consistently better results – between 1.6 and 10.2 better – than the best heuristic on Class 1 from the Mommessin benchmark. The others perform fairly similarly to each other, with the First N Items being slightly worse than the others at no time benefit. As with most other datasets, the unsorted first fit is far worse than the others, but in far less time.

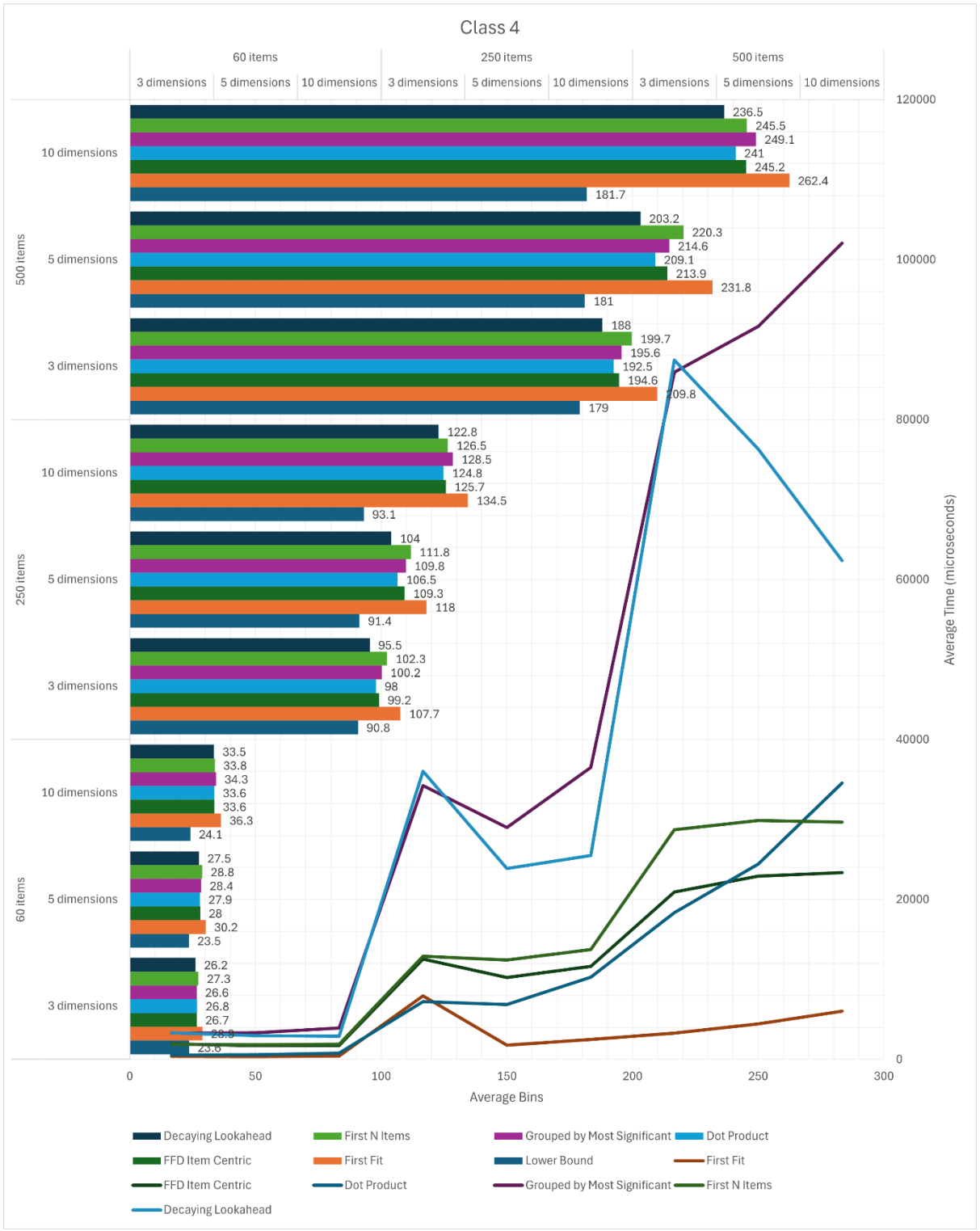


Figure 30 - New Class 4

For the larger instances of this class of benchmarks, the Decaying Lookahead gives a result much better than the other tested heuristics, and the best result for every instance. This remains within a reasonable time, as the time to run on 10 datasets stays under $100000\mu s$.

The Dot Product Heuristic also works well, giving the second best result in all cases and in less time.

8.3.4 Triplet

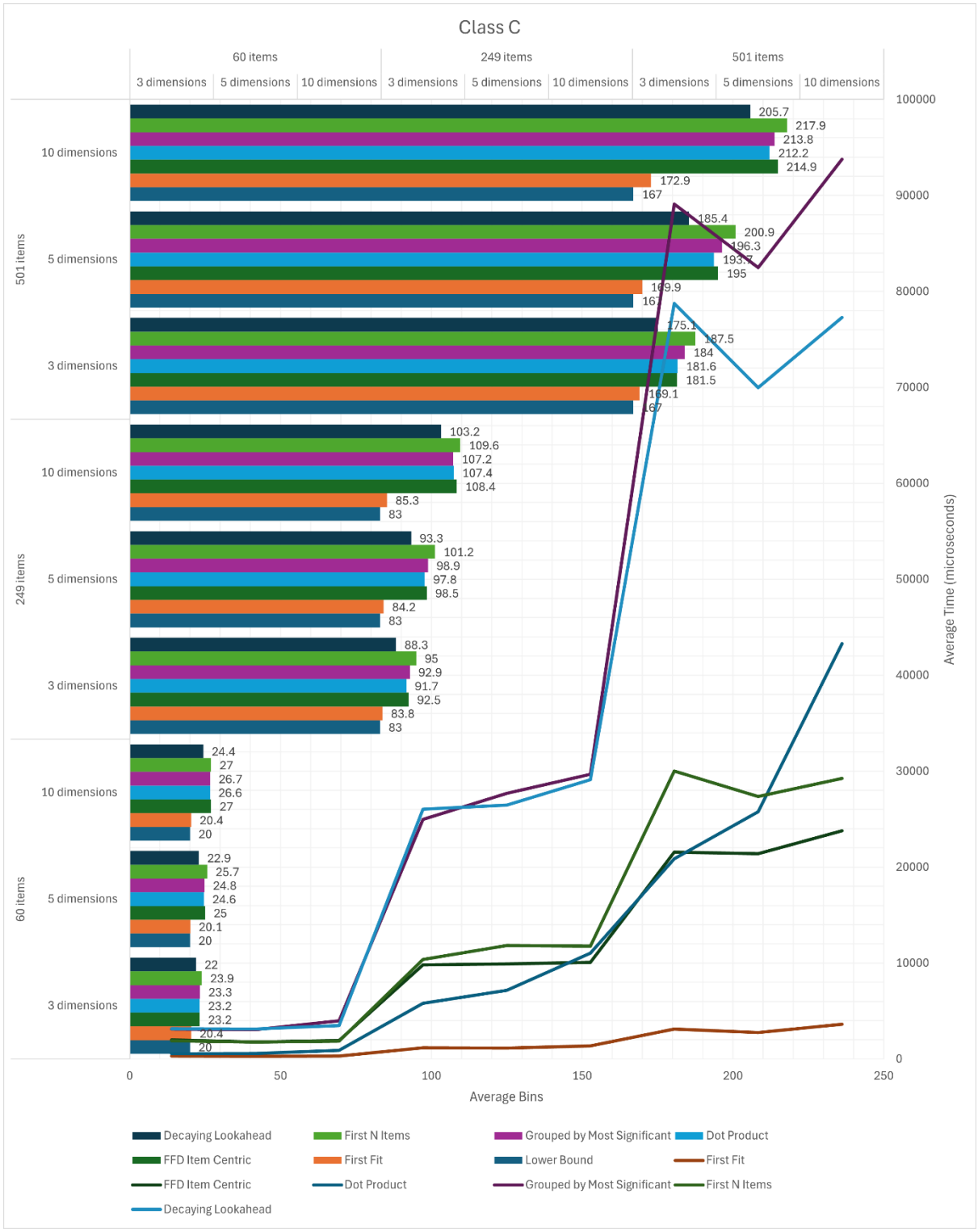


Figure 31 - Caprara Triplet Class

As mentioned in 7.1.4, the first fit gives an unrepresentatively good result on these datasets, as the items are ordered in such a way that they fit very well. Excluding this heuristic, the result is similar to what has been seen with many of the other benchmarks, the Dot Product gives a good balance of time and performance, and the Decaying Lookahead gives a very good result in longer time.

A general recommendation from all benchmarks is that for the best result, Decaying Lookahead is often the best choice as it is closer to the lower bound on average. For a good balance of result and time, Dot Product is a good choice, it is also a more stable heuristic than Decaying Lookahead, giving more consistent and predictable outcomes. Below is a table showing how each heuristic performs:

Heuristic	**Quality of result**	**Speed**	**Stability**
First Fit	Below Average	Very Fast	Very Stable
FFD (L1/L2)	Good	Fast	Very Stable
Dot Product	Very Good	Fast	Stable
First N Items	Acceptable	Moderate	Moderately Stable
Grouped by Most Significant	Inconsistent	Inconsistent	Very Unstable
Decaying Lookahead	Excellent	Slow	Unstable

9. Conclusion

This project researched the Vector Bin Packing Problem, an extension of the classical bin packing problem, where bins and items are represented by a vector. The aim of the project was to investigate how different approaches to the problem were designed, and to propose some new heuristics to find improved solutions to the problem with improvements in the bins used, time taken, or stability of the algorithm. A secondary aim of the project was to investigate and analyse different measures of size of vectors for the first fit decreasing heuristic.

The results demonstrate that the heuristic chosen has a significant impact on the result and the time it takes. Even simple changes can lead to improvements, such as the measure of size used for FFD. It should be noted that all methods of sorting outperformed first fit on an unsorted dataset in most cases. This underlines one of the key principles in VBP of sorting a dataset to better inform the placement of items.

Of the tested measures of size, those based on standard norms – particularly L1 and L2 – gave the most reliable results. These provided a balance between simplicity, and effectiveness, giving a consistently good result across all tested classes of dataset. Some more complicated measures, such as the weighted and multi-factor sorts, performed worse in higher dimensions or larger datasets. This can be attributed to their sensitivity to changes in the nature of the dataset, which may not actually be useful in determining the size of an item. The L∞ norm also showed some inconsistencies in its behaviour, suggesting that relying on only one dimension to determine the size of an item may not be sufficient.

The dot product heuristic, introduced by (Panigrahy, et al., 2011), was one of the most consistently usable heuristics, with reliable speed and consistently good results, overall giving a very stable and effective heuristic. It was not predicted to run in an efficient time, but because it never does a full sort of the items, it can run faster than the item-centric FFD. The heuristic represents a practical balance between quality and efficiency, well suited to real-world applications.

The newly introduced decaying lookahead heuristic was consistently one of the top heuristics from a purely quality of result perspective. In many cases, it provided a significant

improvement over all other tested heuristics. By testing multiple options before deciding on a packing for a bin, it can avoid local optima and pack bins tighter. This comes at a large computational cost. The increased complexity makes it unsuitable for cases that require a stable result or are time sensitive, but if these are not requirements of the use case, then it can be a powerful heuristic to find a very good quality result. If time is not important, the heuristic could be altered to have a higher initial look-forward, or lower decay rate, to test even more options, and increasing the runtime even more.

Other heuristics demonstrated more specialised performance on certain datasets. The grouped by most significant heuristic is a good example of this, with low quality fittings, and slow runtimes on some instances with balanced bins, but the best result on cases with negatively correlated bins. This instability means it is not likely to be a good choice for a more general application, but if the dataset is guaranteed to contain negatively correlated items, it could be used to find a tighter packing.

The first N items heuristic did not perform as well as expected, with the original design idea being that it would provide a less tight packing in a fast time but still outperform the first fit heuristic. However, it provided a slightly unstable result, and was not as fast as some better heuristics, such as the dot product heuristic. There may be ways to make this quicker, for example by placing N items before sorting the dataset, then sorting it after the first N items have been placed, but this would likely give results similar to or worse than first fit.

A key finding is that there is no single heuristic suited to every use case. The priority may differ, with some requiring stability over quality of result, and some prioritising time over everything else. In a general-purpose case where quality of result is the priority, decaying lookahead is the natural choice. If speed is the priority, first fit is the best choice, as it requires no sorting and just places items in the order in which they are input. For a balanced result and stable performance, the dot product heuristic is the best choice, closely followed by the FFD with L1/L2 norms for sorting.

For the FFD algorithm, it would not be unreasonable to run it multiple times with different norms used to minimise the number of bins used. The L1, L2, and L∞ norms would be good choices for this as they are simple, and give good results across a wide range of instances.

The reason running it multiple times is possible is because it is one of the quickest heuristics, and is very stable.

While the datasets used are varied, they may not fully represent real world cases, as the largest dataset contains 500 items. Some cloud platforms will need to place thousands, or even millions, of virtual machines on servers in a datacentre, and instances of this size have not been tested. As a result, the scalability of heuristics cannot be fully determined from this project. Additionally, the datasets used have been systematically generated, and may not accurately represent some real world scenarios.

One non-technical possibility demonstrated in testing cases when items are already in an order such that they fit well. This was shown in the Class C triplet instances, where the first fit algorithm gave the best results, even though it is known to not perform as well as other heuristics such as FFD. This serves as a reminder that understanding the context of the system, and how data is generated can be as important as designing an effective heuristic.

In conclusion, this project has demonstrated that while the VBPP is inherently complex, heuristics can be designed to provide effective solutions for a range of use cases. The results highlight the importance of both item ordering and the way items are placed. By comparing a range of approaches and introducing new heuristics, this project contributes to a deeper understanding of how different strategies perform on the VBPP.

10. Final Comments

In summary, this project has investigated the nature of the vector bin packing problem, and some heuristics used to find an approximate solution in reasonable time. Of those implemented, the two most effective were the existing dot product heuristic (Panigrahy, et al., 2011) and the newly introduced decaying lookahead heuristic. The dot product was more stable, and ran faster, providing a particularly good result, and the decaying lookahead was less stable, but often gave the best results of all tested heuristics. The conclusion from this project is that the requirements of a given case are crucial for choosing a heuristic, and suggestions have been made as to what heuristics are suitable in certain situations.

11. Bibliography

Capara, A. & Toth, P., 2001. Lower bounds and algorithms for the 2-dimensional vector packing problem. *Discrete Applied Mathematics,* 111(3), pp. 231-262.

Csirik, J., Frenk, J., Labbe, M. & Zhang, S., 1990. On the multidimensional vector bin packing. *Acta Cybernetica,* 9(4), pp. 361-369.

Falkenauer, E., 1996. A hybrid grouping genetic algorithm for bin packing.. *Journal of heuristics,* 2(1), pp. 5-30.

Garey, M., Graham, R. & Johnson, D., 1976. Resource constrained scheduling as generalized bin packing. *Journal of Combinatorial Theory, Series A,* 21(3), pp. 257-298.

Kou, L. T. & Markowsky, G., 1977. Multidimensional bin packing algorithms. *IBM Journal of Research and development,* 21(5), pp. 443-448.

Li, C. & Chen, Z.-L., 2006. Bin-packing problem with concave costs of bin utilization.. *Naval Research Logistics (NRL),* 53(4), pp. 298-208.

Mommessin, C., Erlebach, T. & Shakhlevich, N. V., 2025. Classification and evaluation of the algorithms for vector bin packing.. *Computers & Operations Research,* Volume 173.

Nagel, L., Popov, N., Süß, T. & Wang, Z., 2023. Analysis of heuristics for vector scheduling and vector bin packing. In: *Learning and Intelligent Optimization.* s.l.:Cham: Springer, pp. 583-598.

Panigrahy, R., Talwar, K., Uyeda, L. & Wieder, U., 2011. *Heuristics for vector bin packing,* s.l.: research.microsoft.com.

Spieksma, F. C., 1994. A branch-and-bound algorithm for the two-dimensional vector packing problem.. *Computers & operations research,* 21(1), pp. 19-25.